

Weighted Graphs and Applications

Lecture Outline

- *Review* DFS, BFS, Unweighted Shortest Paths 
- Weighted Graphs
- Minimum Spanning Tree (MST)
 - Prim's Algorithm
 - Kruskal's Algorithm
- Weighted Shortest Path Problem
- Dijkstra's Algorithm
 - Definition & Examples
 - Implementing Dijkstra's

DFS

Follow a “choice” all the way to the end, then come back to revisit other choices

```
dfs(Graph graph, Vertex start) {  
    Stack<Vertex> perimeter = new Stack<>();  
    Set<Vertex> visited = new Set<>();  
  
    ...  
}
```

* Can also be implemented recursively; though be careful of stack overflow!

- BFS and DFS are just techniques for iterating! (think: for loop over an array)
 - Need to add code that actually processes something to solve a problem
 - A *lot* of interview problems on graphs can be solved with **modifications on top of BFS or DFS**! Very worth being comfortable with the pseudocode 😊

BFS

Explore layer-by-layer: examine every node at a certain distance from start, then examine nodes that are one level farther

```
bfs(Graph graph, Vertex start) {  
    Queue<Vertex> perimeter = new Queue<>();  
    Set<Vertex> visited = new Set<>();  
  
    perimeter.add(start);  
    visited.add(start);  
  
    while (!perimeter.isEmpty()) {  
        Vertex from = perimeter.remove();  
        for (Edge edge : graph.edgesFrom(from)) {  
            Vertex to = edge.to();  
            if (!visited.contains(to)) {  
                perimeter.add(to);  
                visited.add(to);  
            }  
        }  
    }  
}
```

Review Using BFS for the Shortest Path Problem

(Unweighted) Shortest Path Problem

Given source vertex s and a target vertex t ,
how long is the shortest path from s to t ?
What edges make up that path?

- This is a little harder, but still totally doable! We just need a way to keep track of how far each node is from the start.

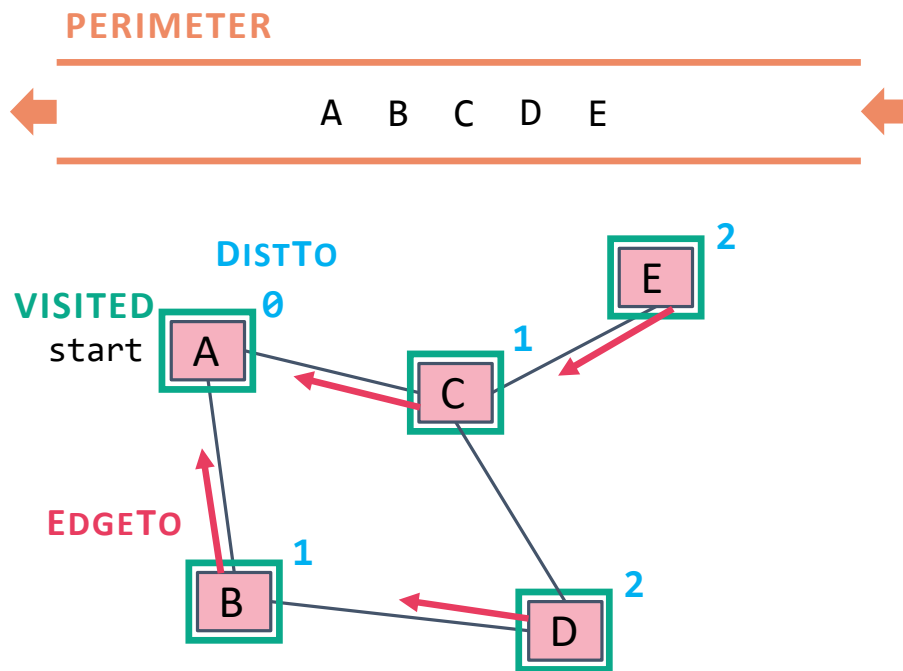
- Sounds like a job for?
 - BFS!

Remember how we got to this point, and what layer this vertex is part of

```
...  
Map<Vertex, Edge> edgeTo = ...  
Map<Vertex, Double> distTo = ...  
  
edgeTo.put(start, null);  
distTo.put(start, 0.0);  
  
while (!perimeter.isEmpty()) {  
    Vertex from = perimeter.remove();  
    for (Edge edge : graph.edgesFrom(from)) {  
        Vertex to = edge.to();  
        if (!visited.contains(to)) {  
            edgeTo.put(to, edge);  
            distTo.put(to, distTo.get(from) + 1);  
            perimeter.add(to);  
            visited.add(to);  
        }  
    }  
}  
return edgeTo;  
}
```

The start required no edge to arrive at, and is on level 0

Review BFS for Shortest Paths: Example

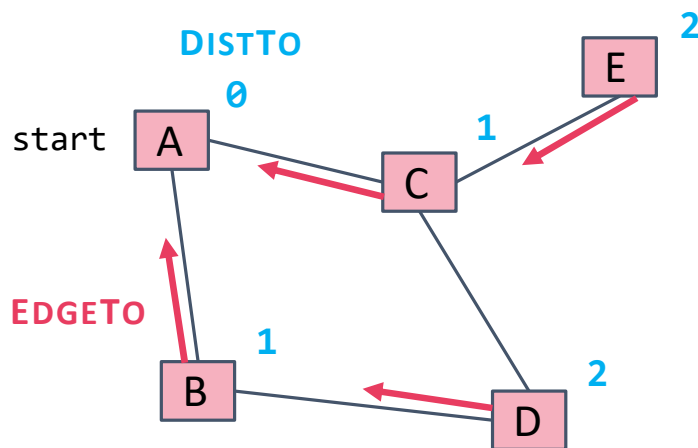


- The edgeTo map stores **backpointers**: each vertex remembers what vertex was used to arrive at it!
- Note: this code stores visited, edgeTo, and distTo as **external maps** (only drawn on graph for convenience). Another implementation option: store them as fields of the nodes themselves

```
...  
Map<Vertex, Edge> edgeTo = ...  
Map<Vertex, Double> distTo = ...  
  
edgeTo.put(start, null);  
distTo.put(start, 0.0);  
  
while (!perimeter.isEmpty()) {  
    Vertex from = perimeter.remove();  
    for (Edge edge : graph.edgesFrom(from)) {  
        Vertex to = edge.to();  
        if (!visited.contains(to)) {  
            edgeTo.put(to, edge);  
            distTo.put(to, distTo.get(from) + 1);  
            perimeter.add(to);  
            visited.add(to);  
        }  
    }  
}  
return edgeTo;  
}
```

Review What about the Target Vertex?

Shortest Path Tree:



- This modification on BFS didn't mention the target vertex at all!
- Instead, it calculated the shortest path and distance from start to *every other vertex*
 - This is called the **shortest path tree**
 - A general concept: in this implementation, made up of **distances** and **backpointers**
- Shortest path tree has all the answers!
 - **Length of shortest path from A to D?**
 - Lookup in **distTo** map: **2**
 - **What's the shortest path from A to D?**
 - Build up backwards from **edgeTo** map: start at D, follow **backpointer** to B, follow **backpointer** to A – our shortest path is **A → B → D**
- All our shortest path algorithms will have this property
 - If you only care about t, you can sometimes stop early!

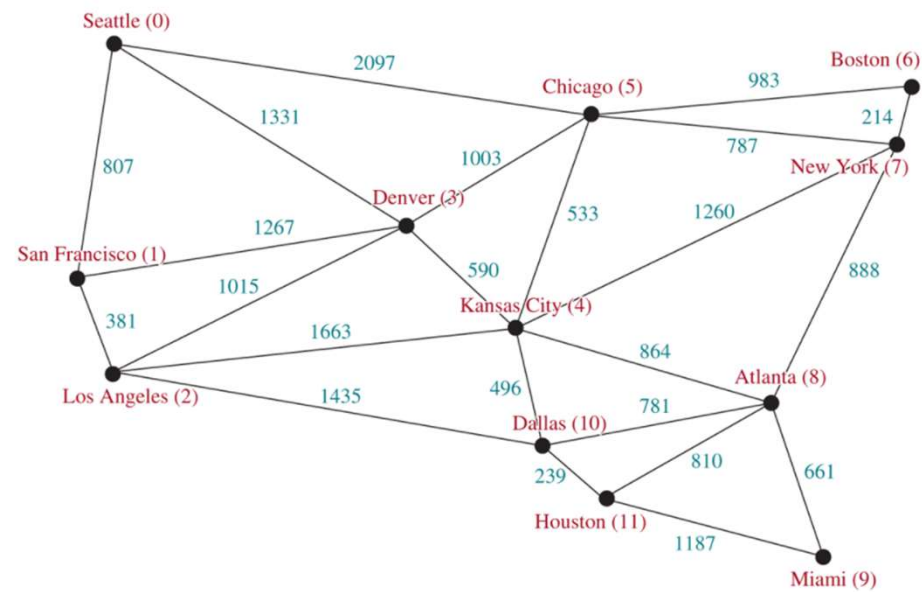
Lecture Outline

- *Review* DFS, BFS, Unweighted Shortest Paths

- **Weighted Graphs** 

- Minimum Spanning Tree (MST)
 - Prim's Algorithm
 - Kruskal's Algorithm
- Weighted Shortest Path Problem
- Dijkstra's Algorithm
 - Definition & Examples
 - Implementing Dijkstra's

Weighted Graph

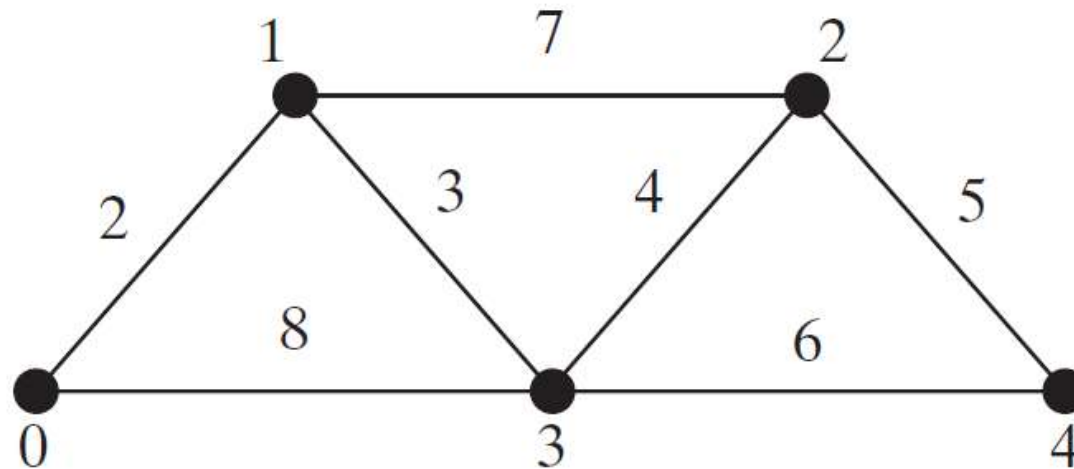


Representing Weighted Graphs

Representing Weighted Edges: Edge Array

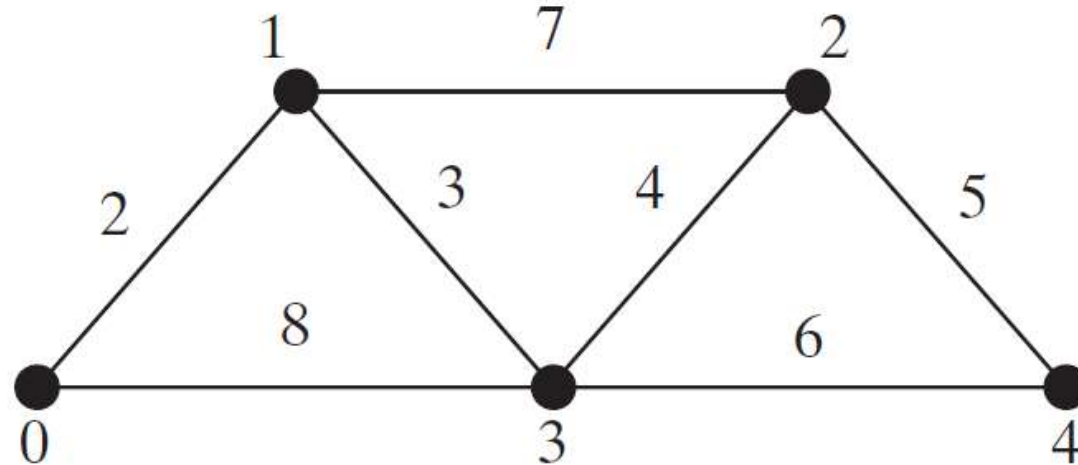
Weighted Adjacency Matrices

Adjacency Lists



Representing Weighted Edges: Edge Array

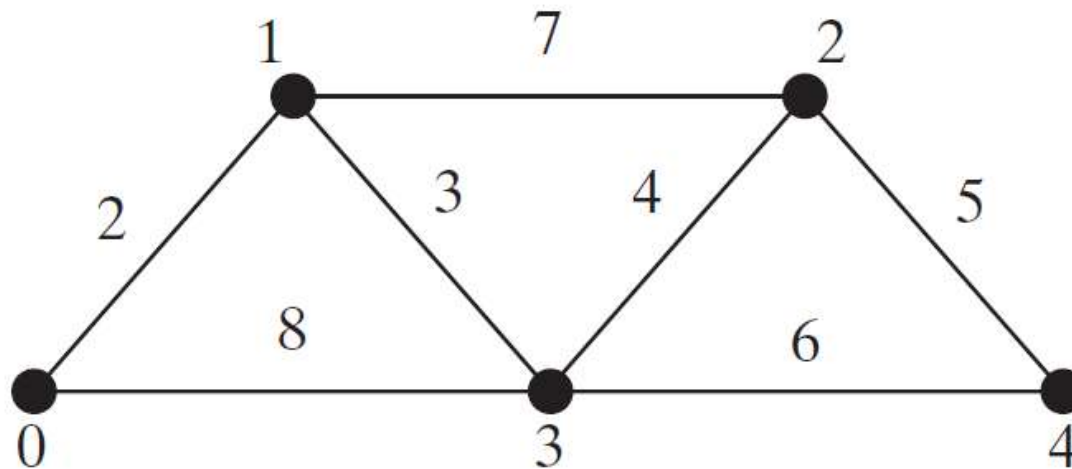
```
int[][] edges = {{0, 1, 2}, {0, 3, 8},  
                {1, 0, 2}, {1, 2, 7}, {1, 3, 3},  
                {2, 1, 7}, {2, 3, 4}, {2, 4, 5},  
                {3, 0, 8}, {3, 1, 3}, {3, 2, 4}, {3, 4, 6},  
                {4, 2, 5}, {4, 3, 6}};  
};
```



Representing Weighted Edges: Edge Array

```
Integer[][] adjacencyMatrix = {  
    {null, 2, null, 8, null },  
    {2, null, 7, 3, null },  
    {null, 7, null, 4, 5},  
    {8, 3, 4, null, 6},  
    {null, null, 5, 6, null}
```

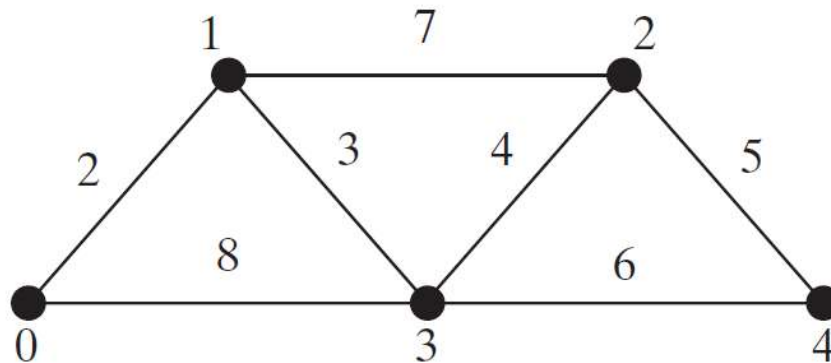
	0	1	2	3	4
0	null	2	null	8	null
1	2	null	7	3	null
2	null	7	null	4	5
3	8	3	4	null	6
4	null	null	5	6	null



Edge Adjacency Lists

```
java.util.List<WeightedEdge>[] neighbors = new java.util.List[5];
```

neighbors[0]	WeightedEdge(0, 1, 2)	WeightedEdge(0, 3, 8)	
neighbors[1]	WeightedEdge(1, 0, 2)	WeightedEdge(1, 3, 3)	WeightedEdge(1, 2, 7)
neighbors[2]	WeightedEdge(2, 3, 4)	WeightedEdge(2, 4, 5)	WeightedEdge(2, 1, 7)
neighbors[3]	WeightedEdge(3, 1, 3)	WeightedEdge(3, 2, 4)	WeightedEdge(3, 4, 6)
neighbors[4]	WeightedEdge(4, 2, 5)	WeightedEdge(4, 3, 6)	



What is the output of the following code?

```
List<WeightedEdge> list = new ArrayList<>();  
list.add(new WeightedEdge(1, 2, 3.5));  
list.add(new WeightedEdge(2, 3, 4.5));  
WeightedEdge e = java.util.Collections.max(list);  
System.out.println(e.u);  
System.out.println(e.v);  
System.out.println(e.weight);
```

If a priority queue is used to store weighted edges, what is the output of the following code?

```
PriorityQueue<WeightedEdge> q = new PriorityQueue<>();  
q.offer(new WeightedEdge(1, 2, 3.5));  
q.offer(new WeightedEdge(1, 6, 6.5));  
q.offer(new WeightedEdge(1, 7, 1.5));  
System.out.println(q.poll().weight);  
System.out.println(q.poll().weight);  
System.out.println(q.poll().weight);
```

- If a priority queue is used to store weighted edges, what is wrong in the following code? Fix it and show the output.

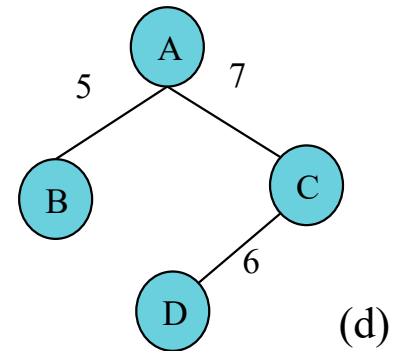
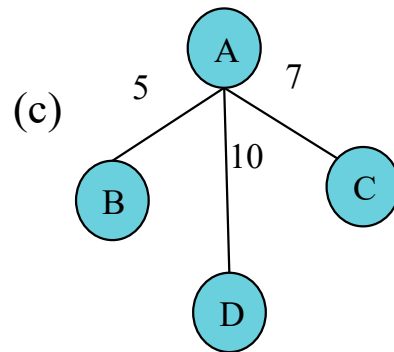
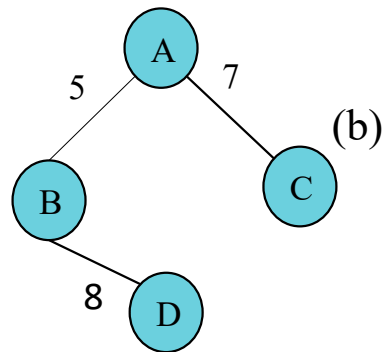
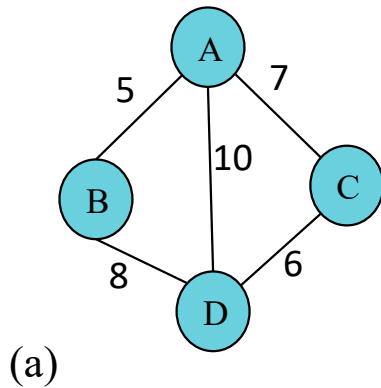
```
1. List<PriorityQueue<WeightedEdge>> queues = new ArrayList<>();
2. queues.get(0).offer(new WeightedEdge(0, 2, 3.5));
3. queues.get(0).offer(new WeightedEdge(0, 6, 6.5));
4. queues.get(0).offer(new WeightedEdge(0, 7, 1.5));
5. queues.get(1).offer(new WeightedEdge(1, 0, 3.5));
6. queues.get(1).offer(new WeightedEdge(1, 5, 8.5));
7. queues.get(1).offer(new WeightedEdge(1, 8, 19.5));
8. System.out.println(queues.get(0).peek().compareTo(queues.get(1).peek()));
```

Lecture Outline

- *Review* DFS, BFS, Unweighted Shortest Paths
- Weighted Graphs
- **Minimum Spanning Tree (MST)** 
 - Prim's Algorithm
 - Kruskal's Algorithm
- Weighted Shortest Path Problem
- Dijkstra's Algorithm
 - Definition & Examples
 - Implementing Dijkstra's

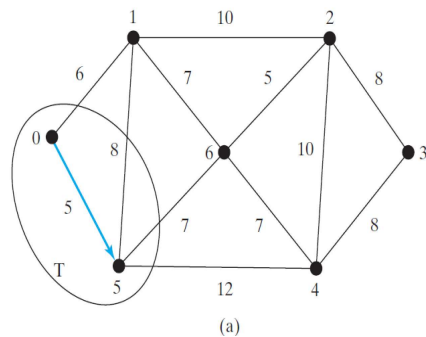
Spanning Trees

- A Spanning tree of a connected undirected graph G is a subgraph of G that contains all of G 's vertices and enough of its edges to form a tree.
- A spanning tree is subgraph of G that is connected with no cycles
- A graph may have many spanning trees.

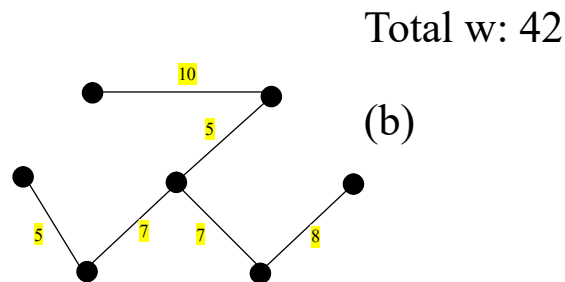


Minimum Spanning Trees

A graph may have many spanning trees. Suppose that the edges are weighted. A minimum spanning tree is a spanning tree with the minimum total weights. For example, the trees in Figures (b), (c), (d) are spanning trees for the graph in Figure (a). The trees in Figures (c) and (d) are minimum spanning trees.



(a)

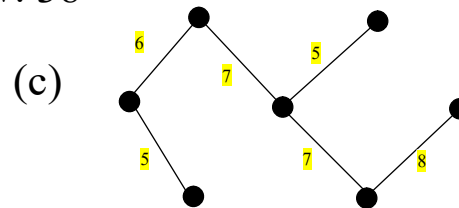


Total w: 42

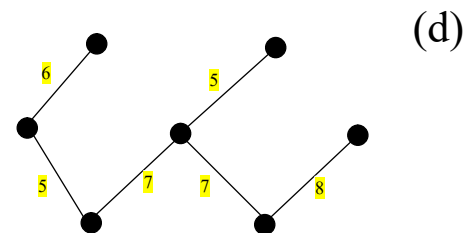
(b)

Total w: 38

Total w: 38



(c)



(d)

MST

- Given a connected and undirected graph, a spanning tree of that graph is a subgraph that is a tree and connects all the vertices together.
- A MST has $(V-1)$ edges where V is the number of vertices in the given graph.

Lecture Outline

- *Review* DFS, BFS, Unweighted Shortest Paths
- Weighted Graphs
- Minimum Spanning Tree (MST)
 - Prim's Algorithm 
 - Kruskal's Algorithm
- Weighted Shortest Path Problem
- Dijkstra's Algorithm
 - Definition & Examples
 - Implementing Dijkstra's

Prim's Minimum Spanning Tree Algorithm

Input: $G = (V, E)$ with non-negative weights.

Output: a MST

MST minimumSpanningTree(sv) { // sv $\leftarrow$ startVertex

Let V denote the set of vertices in the graph;

Let T be a set for the vertices in the spanning tree;

Let MST be a set for the edges in the spanning tree;

Initially, add the sv to T ;

Add edgeList of sv to a collection (What kind? Later)

while (size of $T < n$) {

find v in T and u in $V - T$ with the smallest weight
on the edge (u, v) , as shown in the figure;

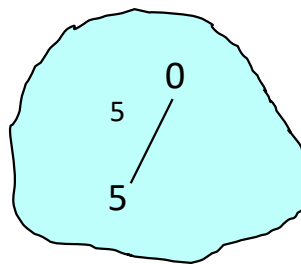
add u to T ; add edge to MST

add edgeList of u to collection

}

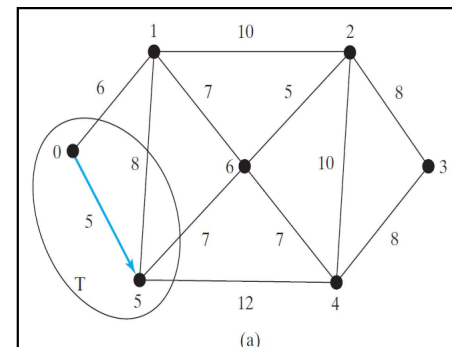
}

MST

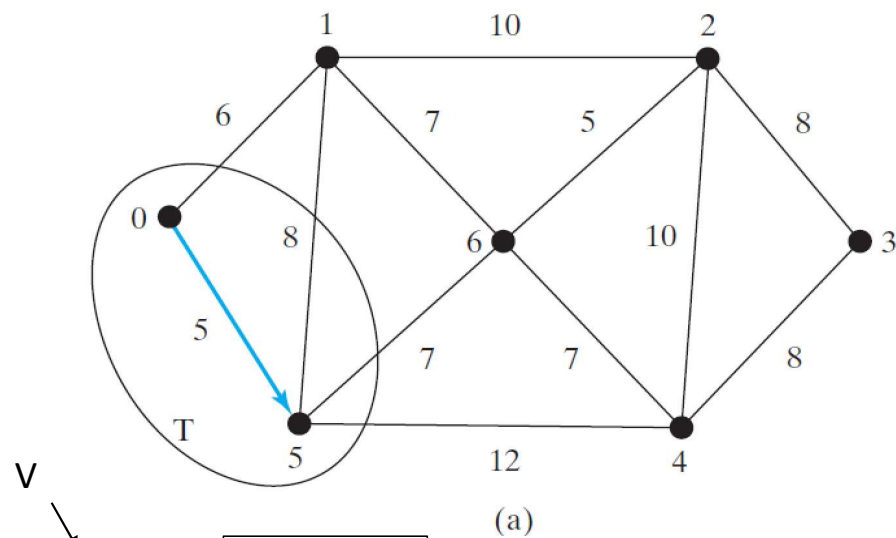


Find a vertex in $V-T$ that has the smallest weighted edge connecting to a vertex in T .

$V-T$



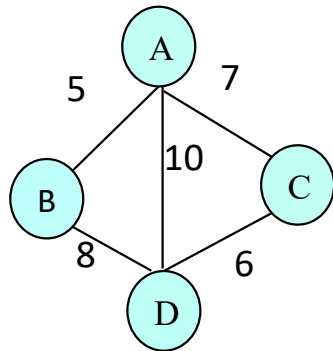
MST Example



V

0	→	{0,1,6} {0,5,5}
1	→	{1,0,6} {1,2,10} {1,5,8} {1,6,7}
2	→	{2,1,10} {2,3,8} {2,4,10} {2,6,5}
3	→	{3,2,8} {3,4,8}
4	→	{4,2,10} {4,3,8} {4,5,12} {4,6,7}
5	→	{5,0,5} {5,1,8} {5,4,12} {5,6,7}
6	→	{6,1,7} {6,2,5} {6,4,7} {6,5,7}

MST Example



A	→	(A,B,5), (A,C,7), (A,D,10)
B	→	(B,A,5), (B,D,8)
C	→	(C,A,7), (C,D,6)
D	→	(D,A,10), (D,B,8), (D,C,6)

Suppose we start with B

MST {}

MST { <B,A,5> }

MST { <B,A,5>, <A,C,7> }

MST { <B,A,5>, <A,C,7>, <C,D,6> }

<u>I</u>	<u>PQ</u>
B	<B,A,5> <B,D,8>
A	<A,C,7> <B,D,8> <A,D,10>
C	<C,D,6> <B,D,8> <A,D,10>
D	<B,D,8> <A,D,10>

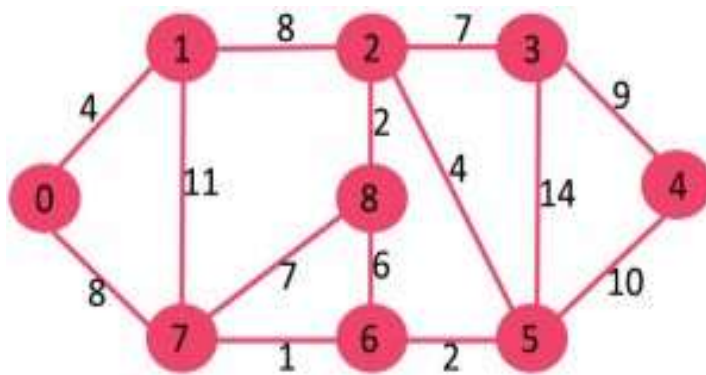
Lecture Outline

- *Review* DFS, BFS, Unweighted Shortest Paths
- Weighted Graphs
- Minimum Spanning Tree (MST)
 - Prim's Algorithm
 - **Kruskal's Algorithm** 
- Weighted Shortest Path Problem
- Dijkstra's Algorithm
 - Definition & Examples
 - Implementing Dijkstra's

Kruskal's MST

1. Sort all the edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far.
 - If (cycle is not formed)
 - include this edge in MST
 - else
 - discard it.
3. Repeat step # 2 until there are $(V-1)$ edges in the spanning tree.

Kruskal's MST



- After sorting

Weight	src	dest
1	7	6
2	8	2
2	6	5
4	0	1
4	2	5
6	8	6
7	2	3
7	7	8
8	0	7
8	1	2
9	3	4
10	5	4
11	1	7
14	3	5

Step # 2 (Cycle Detection)

Union-Find algorithm

- A disjoint-set data structure is a data structure that keeps track of a set of elements partitioned into a number of disjoint (non-overlapping) subsets.
- A union-find algorithm is an algorithm that performs two useful operations on such a data structure:
 - **Find:** Determine which subset a particular element is in. This can be used for determining if two elements are in the same subset.
 - **Union:** Join two subsets into a single subset.

Lecture Outline

- *Review* DFS, BFS, Unweighted Shortest Paths
- Weighted Graphs
- Minimum Spanning Tree (MST)
 - Prim's Algorithm
 - Kruskal's Algorithm
- **Weighted Shortest Path Problem** 
- Dijkstra's Algorithm
 - Definition & Examples
 - Implementing Dijkstra's

Our Graph Problem Collection

s-t Connectivity Problem

Given source vertex s and a target vertex t , does there exist a path from s to t ?

SOLUTION

Base Traversal: BFS or DFS
Modification: Check if each vertex == t

Unweighted Shortest Path Problem

Given source vertex s and target vertex t , what path from s to t minimizes the number of edges? How long is that path, and what edges make it up?

SOLUTION

Base Traversal: BFS
Modification: Generate shortest path tree as we go

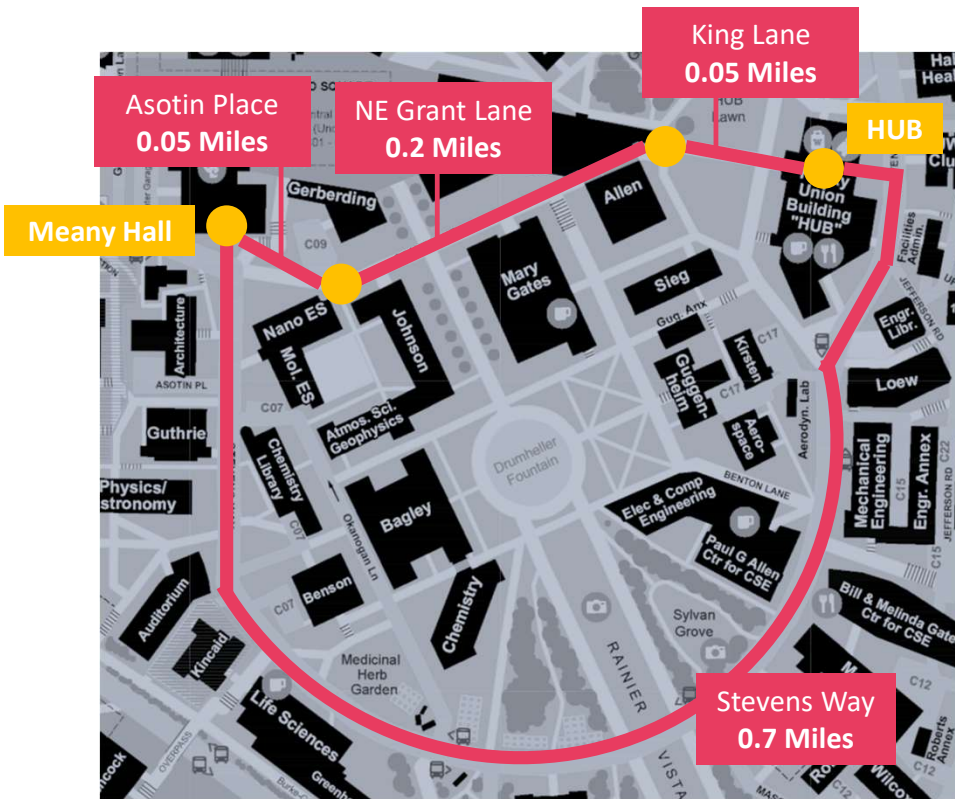
NEW

Weighted Shortest Path Problem

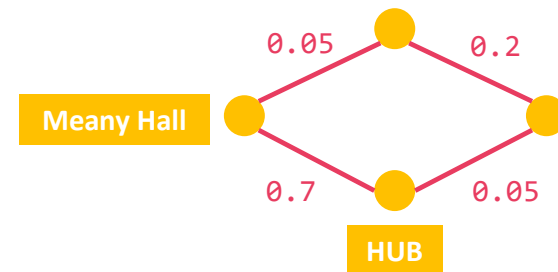
Given source vertex s and target vertex t , what path from s to t minimizes the total weight of its edges? How long is that path, and what edges make it up?

???

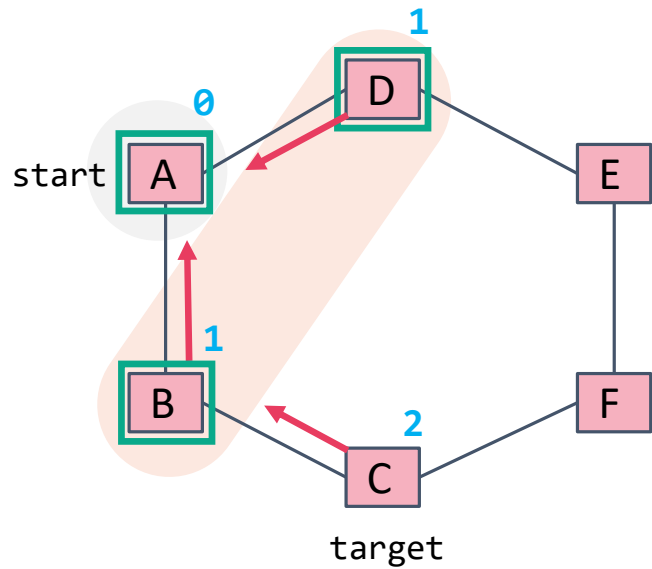
The Weighted Shortest Path Problem



- It's lunchtime, and that Pagliacci slice isn't going to eat itself – Suppose we want to find the fastest path from Meany Hall to the HUB
 - Model as a graph: buildings & road meeting points are vertices, roads are edges
- Of course, want to take Asotin – Grant – King, not Stevens Way!
 - Use edge weights to model distance, since *not all edges have the same cost*
 - Would BFS give us the right answer here?



Why does BFS work for unweighted graphs?



Observation: The “First Try Phenomenon”

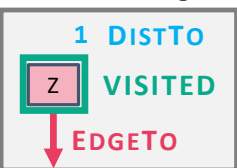
- BFS only enqueues each vertex once (makes it efficient)
- As soon as BFS enqueues a vertex, the final path to that vertex has been chosen! Never re-evaluate its path.



Key Intuition: BFS works because:

- IF we always process the closest vertices first,
- THEN the first path we discover to a new vertex will *always be the shortest!*

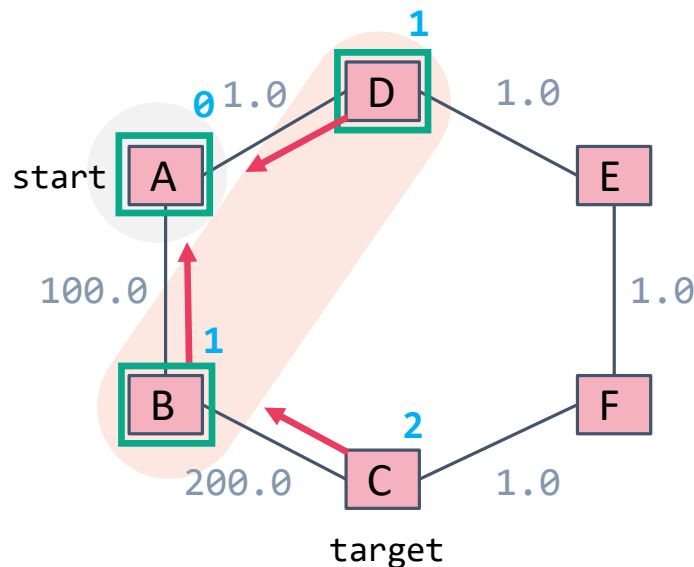
BFS Tracking:



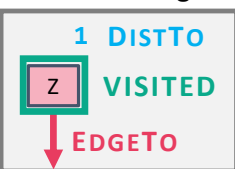
Example: For shortest path to C, why do we choose edge (B,C) and not (F,C)?

- Exactly *because* we visit B before F!

Why *doesn't* BFS work for weighted graphs?



BFS Tracking:



- We want the path that minimizes the sum of edge weights
 - A-D-E-F-C: total distance 4
 - A-B-C: total distance 300.
- Do the edge weights affect how BFS runs?
 - Nope! Exactly the same path chosen
- **Observation:** still have “First Try Phenomenon”
- **Key Intuition:** yet BFS breaks because we no longer process the closest vertices first (that is, not closest according to the edge weights!)
 - So we can’t rely on first path found being best anymore 😞
- **Idea 1:** Could we change the weighted graph into an unweighted graph?

Lecture Outline

- *Review* DFS, BFS, Unweighted Shortest Paths
- Weighted Graphs
- Minimum Spanning Tree (MST)
 - Prim's Algorithm
 - Kruskal's Algorithm
- Weighted Shortest Path Problem
- **Dijkstra's Algorithm**
 - **Definition & Examples** 
 - Implementing Dijkstra's

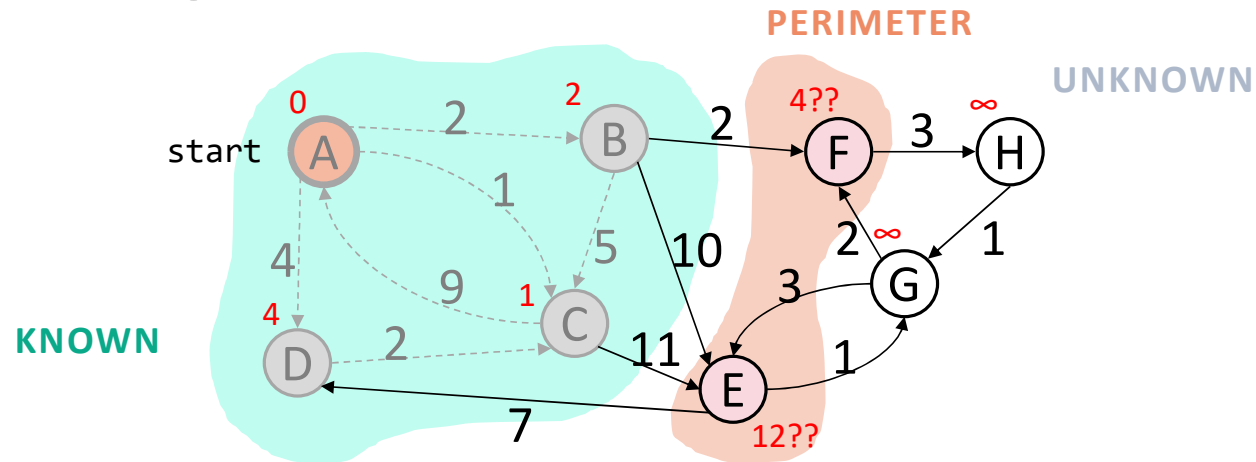
Lecture Outline

- **Review** DFS, BFS, Unweighted Shortest Paths
- Weighted Shortest Path Problem
- Reductions: Weighted $\rightarrow$ Unweighted
- **Dijkstra's Algorithm**
 - Definition & Examples 
 - Implementing Dijkstra's

Dijkstra's Algorithm

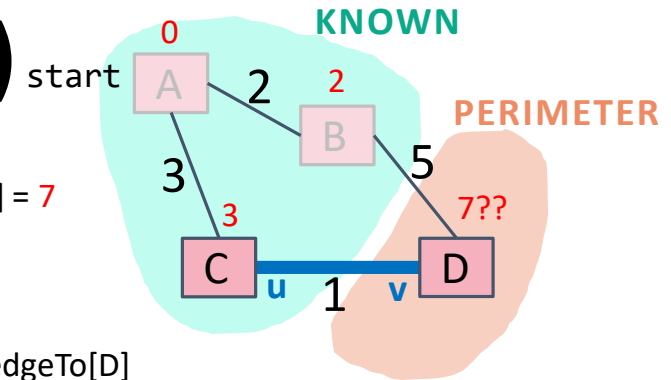
- Named after its inventor, Edsger Dijkstra (1930-2002)
 - Truly one of the “founders” of computer science
 - 1972 Turing Award
 - This algorithm is just *one* of his many contributions!
 - Example quote: “Computer science is no more about computers than astronomy is about telescopes”
- The idea: reminiscent of BFS, but adapted to handle weights
 - Grow the set of nodes whose shortest distance has been computed
 - Nodes not in the set will have a “best distance so far”

Dijkstra's Algorithm: Idea



- Initialization:
 - Start vertex has distance **0**; all other vertices have distance ∞
- At each step:
 - Pick closest unknown vertex v
 - Add it to the “cloud” of known vertices
 - Update “best-so-far” distances for vertices with edges from v

Dijkstra's Pseudocode (High-Level)



- Suppose we already visited B, $\text{distTo}[D] = 7$
- Now considering edge (C, D):
 - $\text{oldDist} = 7$
 - $\text{newDist} = 3 + 1$
 - That's better! Update $\text{distTo}[D]$, $\text{edgeTo}[D]$

Similar to “visited” in BFS, “known” is nodes that are finalized (we know their path)

Dijkstra's algorithm is all about updating “best-so-far” in distTo if we find shorter path! Init all paths to infinite.

Order matters: always visit closest first!

Consider all vertices reachable from me: would getting there *through* me be a shorter path than they currently know about?

```
dijkstraShortestPath(G graph, V start)
```

```
Set known; Map edgeTo, distTo;
```

```
initialize distTo with all nodes mapped to  $\infty$ , except start to 0
```

```
while (there are unknown vertices):
```

```
let u be the closest unknown vertex
```

```
known.add(u);
```

```
for each edge (u,v) from u with weight w:
```

```
oldDist = distTo.get(v) // previous best path to v
```

```
newDist = distTo.get(u) + w // what if we went through u?
```

```
if (newDist < oldDist):
```

```
distTo.put(v, newDist)
```

```
edgeTo.put(v, u)
```

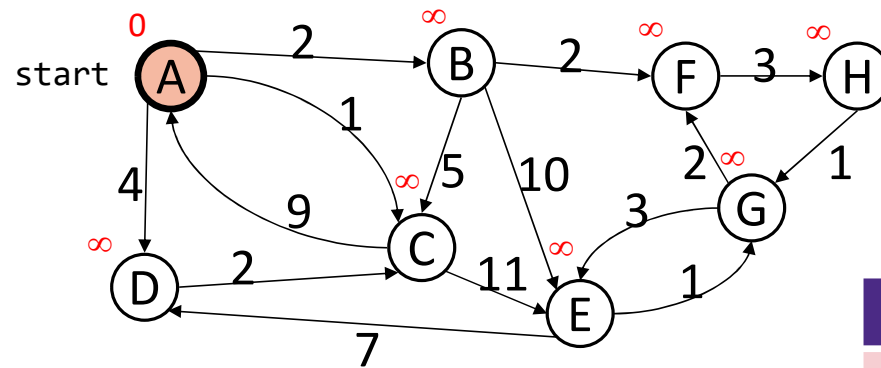
Dijkstra's Algorithm: Key Properties

- Once a vertex is marked known, its shortest path is known
 - Can reconstruct path by following back-pointers (in edgeTo map)
- While a vertex is not known, another shorter path might be found
 - We call this update **relaxing** the distance because it only ever shortens the current best path
- Going through closest vertices first lets us confidently say no shorter path will be found once known
 - Because not possible to find a shorter path that uses a farther vertex we'll consider later

```
dijkstraShortestPath(G graph, V start)
    Set known; Map edgeTo, distTo;
    initialize distTo with all nodes mapped to  $\infty$ , except start to 0

    while (there are unknown vertices):
        let u be the closest unknown vertex
        known.add(u)
        for each edge (u,v) to unknown v with weight w:
            oldDist = distTo.get(v)           // previous best path to v
            newDist = distTo.get(u) + w       // what if we went through u?
            if (newDist < oldDist):
                distTo.put(v, newDist)
                edgeTo.put(v, u)
```

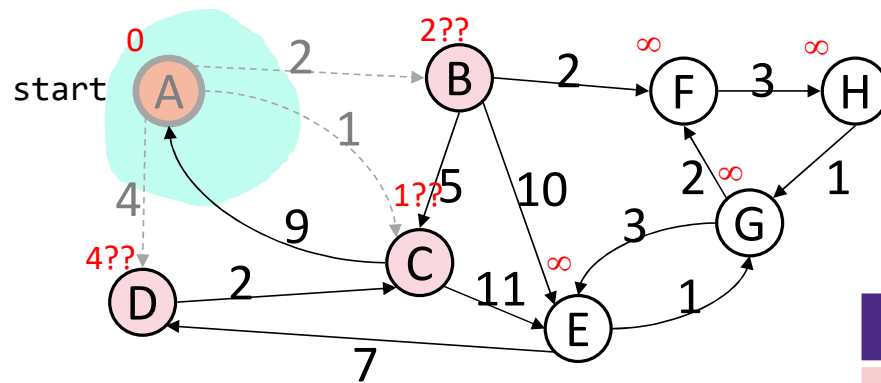
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:

Vertex	Known?	distTo	edgeTo
A			
B			
C			
D			
E			
F			
G			
H			

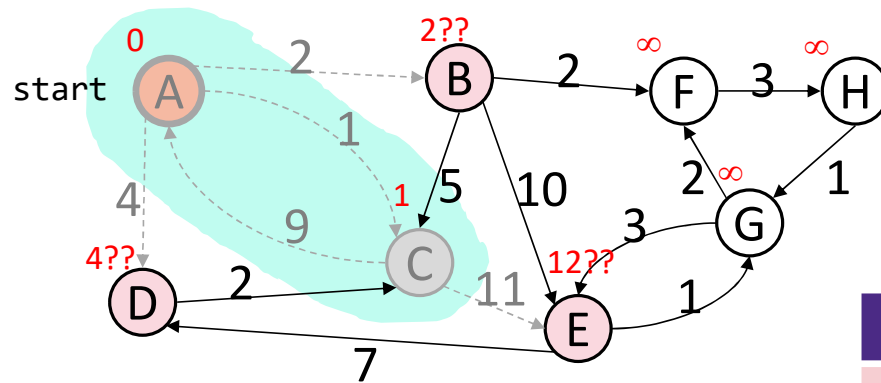
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:
A

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 2	A
C		≤ 1	A
D		≤ 4	A
E		∞	
F		∞	
G		∞	
H		∞	

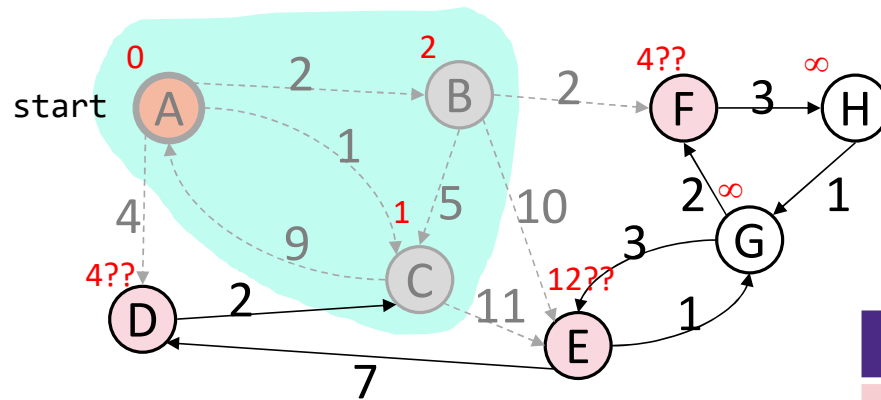
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:
A, C

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 2	A
C	Y	1	A
D		≤ 4	A
E		≤ 12	C
F		∞	
G		∞	
H		∞	

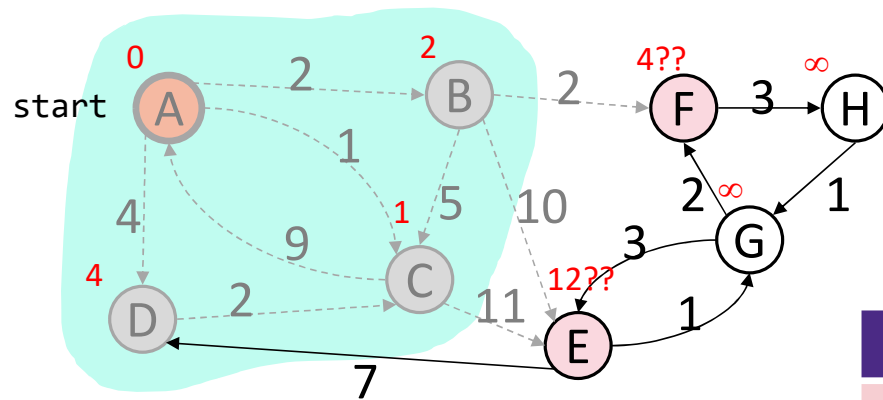
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:
A, C, B

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D		≤ 4	A
E		≤ 12	C
F		≤ 4	B
G		∞	
H		∞	

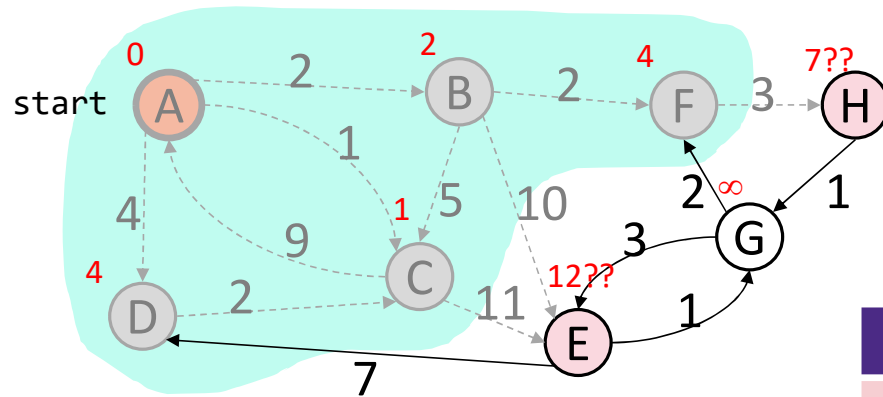
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:
A, C, B, D

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F		≤ 4	B
G		∞	
H		∞	

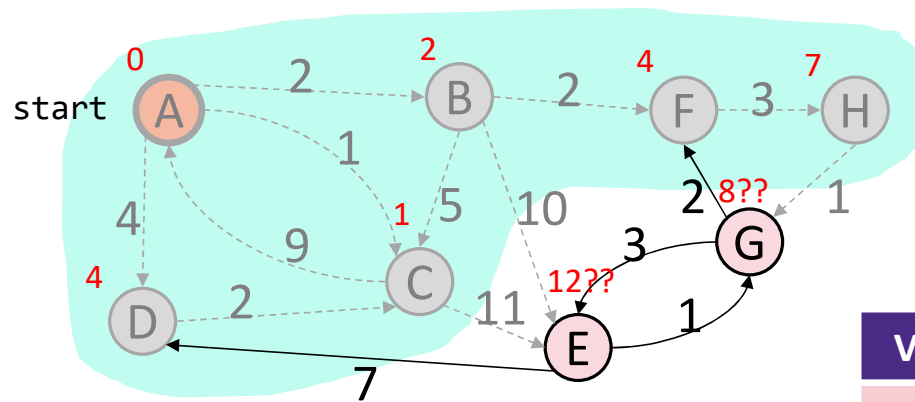
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:
A, C, B, D, F

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F	Y	4	B
G		∞	
H		≤ 7	F

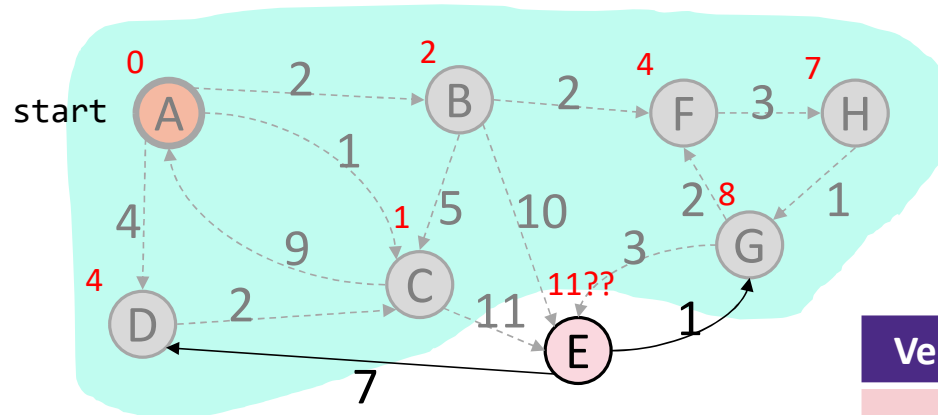
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:
A, C, B, D, F, H

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 12	C
F	Y	4	B
G		≤ 8	H
H	Y	7	F

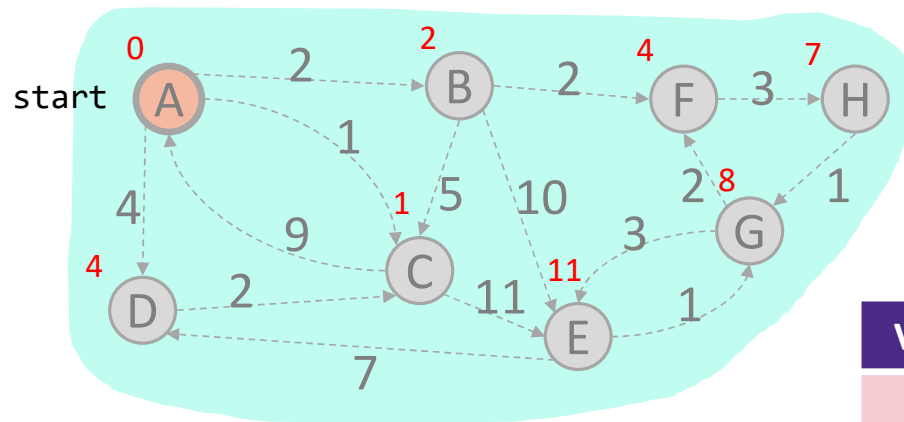
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:
A, C, B, D, F, H, G

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E		≤ 11	G
F	Y	4	B
G	Y	8	H
H	Y	7	F

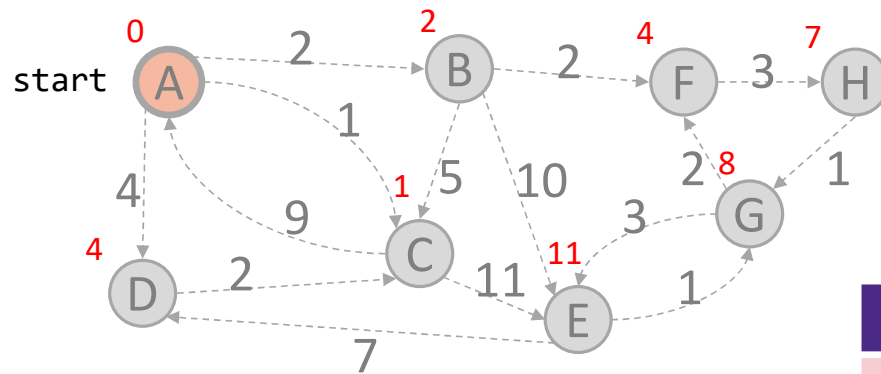
Dijkstra's Algorithm: Example #1



Order Added to
Known Set:
A, C, B, D, F, H, G, E

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E	Y	11	G
F	Y	4	B
G	Y	8	H
H	Y	7	F

Dijkstra's Algorithm: Interpreting the Results



Now that we're done, how do we get the path from A to E?

- Follow edgeTo backpointers!
- distTo and edgeTo make up the **shortest path tree**

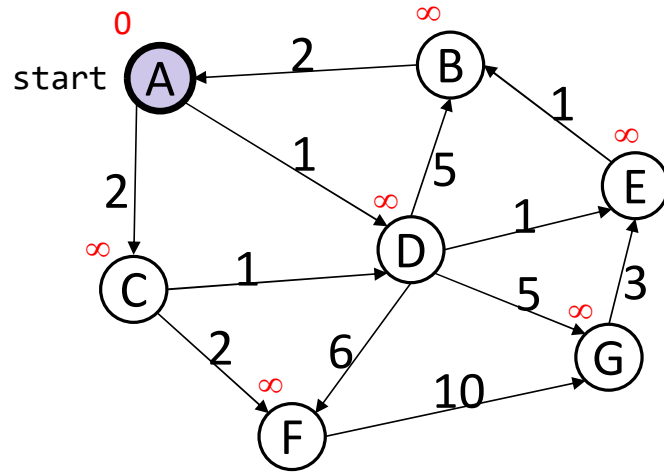
Order Added to
Known Set:
A, C, B, D, F, H, G, E

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	2	A
C	Y	1	A
D	Y	4	A
E	Y	11	G
F	Y	4	B
G	Y	8	H
H	Y	7	F

Review: Key Features

- Once a vertex is marked known, its shortest path is known
 - Can reconstruct path by following backpointers
- While a vertex is not known, another shorter path might be found!
- The “Order Added to Known Set” is unimportant
 - A detail about how the algorithm works (*client doesn't care*)
 - Not used by the algorithm (*implementation doesn't care*)
 - It is sorted by path-distance; ties are resolved “somehow”
- If we only need path to a specific vertex, can stop early once that vertex is known
 - Because its shortest path cannot change!
 - Return a partial **shortest path tree**

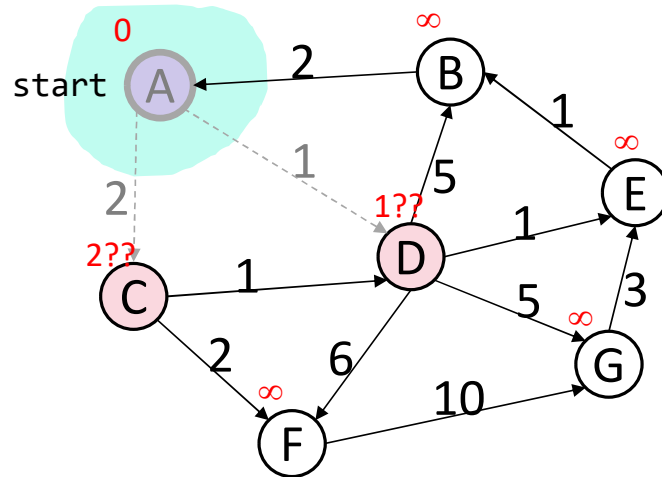
Dijkstra's Algorithm: Example #2



Order Added to
Known Set:

Vertex	Known?	distTo	edgeTo
A		∞	
B		∞	
C		∞	
D		∞	
E		∞	
F		∞	
G		∞	

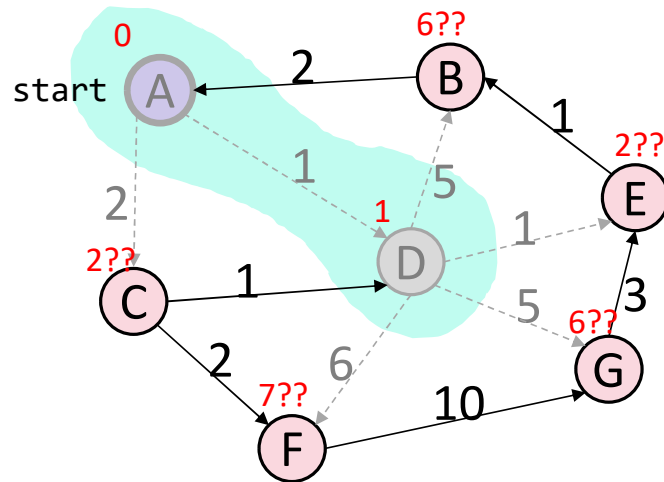
Dijkstra's Algorithm: Example #2



Order Added to
Known Set:
A

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		∞	
C		≤ 2	A
D		≤ 1	A
E		∞	
F		∞	
G		∞	

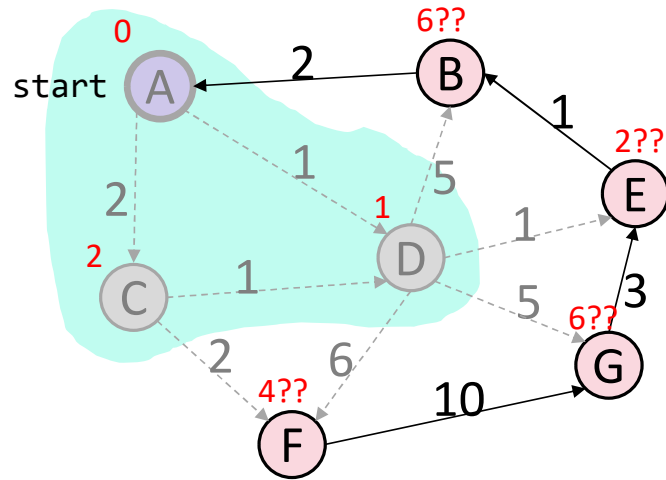
Dijkstra's Algorithm: Example #2



Order Added to
Known Set:
A, D

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 6	D
C		≤ 2	A
D	Y	1	A
E		≤ 2	D
F		≤ 7	D
G		≤ 6	D

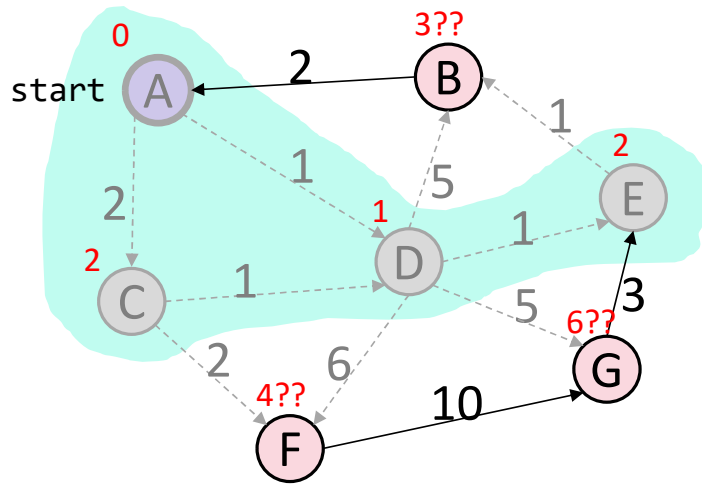
Dijkstra's Algorithm: Example #2



Order Added to
Known Set:
A, D, C

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 6	D
C	Y	2	A
D	Y	1	A
E		≤ 2	D
F		≤ 4	C
G		≤ 6	D

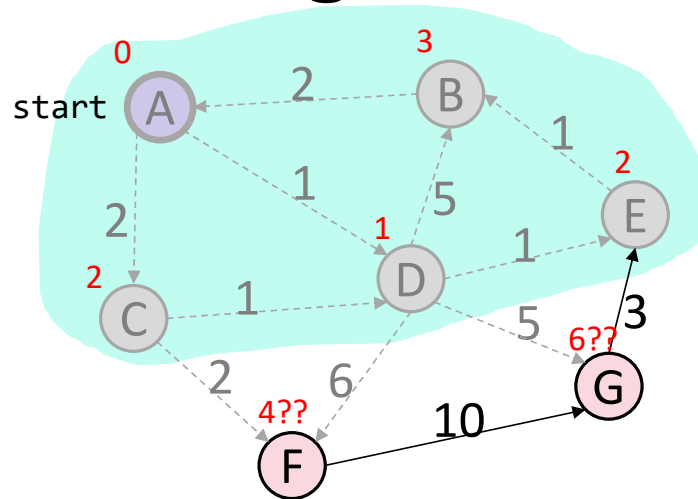
Dijkstra's Algorithm: Example #2



Order Added to
Known Set:
A, D, C, E

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F		≤ 4	C
G		≤ 6	D

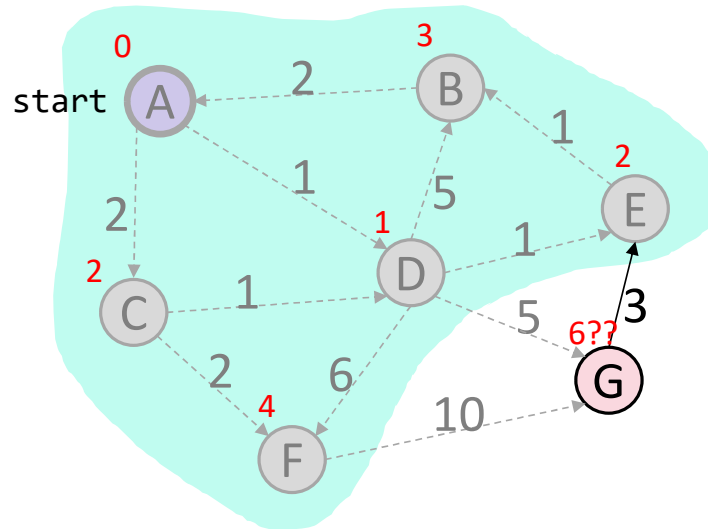
Dijkstra's Algorithm: Example #2



Order Added to
Known Set:
A, D, C, E, B

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F		≤ 4	C
G		≤ 6	D

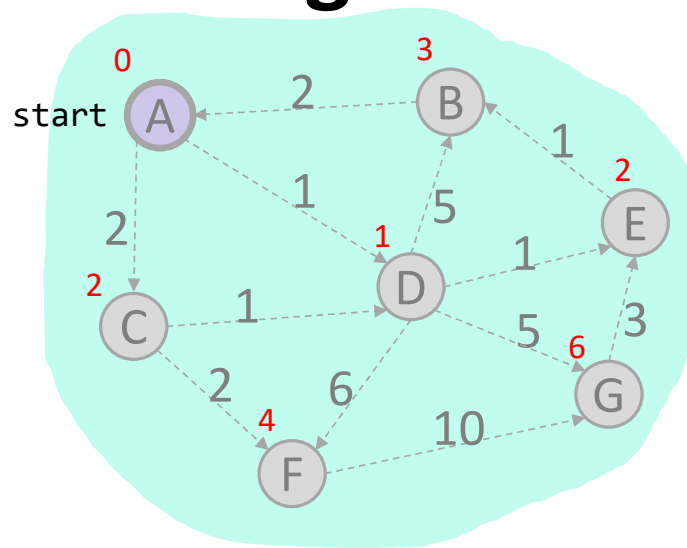
Dijkstra's Algorithm: Example #2



Order Added to
Known Set:
A, D, C, E, B, F

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F	Y	4	C
G		≤ 6	D

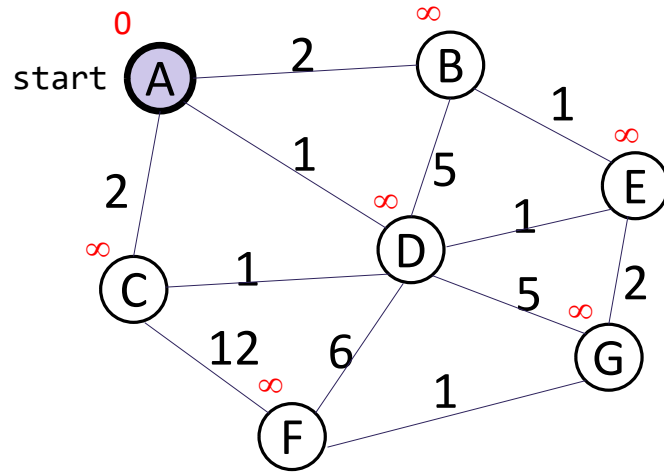
Dijkstra's Algorithm: Example #2



Order Added to
Known Set:
A, D, C, E, B, F, G

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	3	E
C	Y	2	A
D	Y	1	A
E	Y	2	D
F	Y	4	C
G	Y	6	D

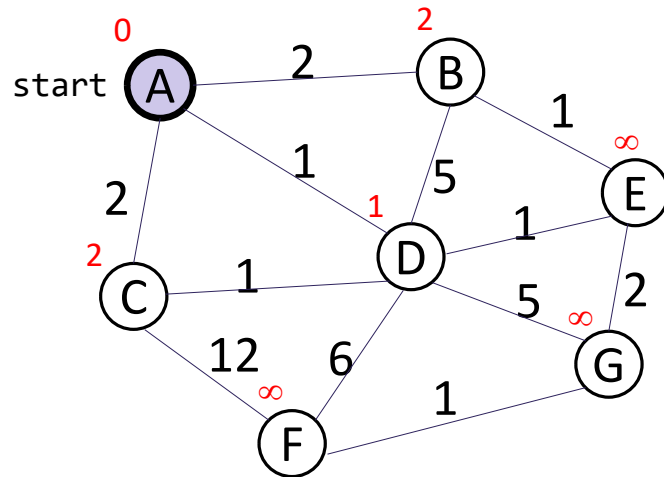
Dijkstra's Algorithm: Example #3 (undirected)



Order Added to
Known Set:

Vertex	Known?	distTo	edgeTo
A		∞	
B		∞	
C		∞	
D		∞	
E		∞	
F		∞	
G		∞	

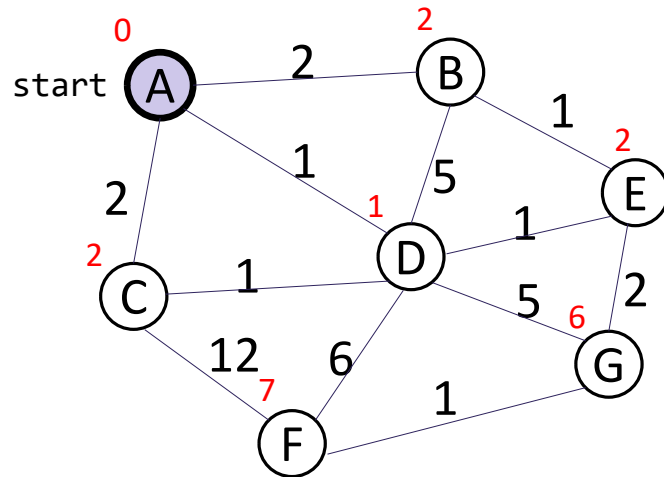
Dijkstra's Algorithm: Example #3 (undirected)



Order Added to
Known Set:
A,

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 2	A
C		≤ 2	A
D		≤ 1	A
E		∞	
F		∞	
G		∞	

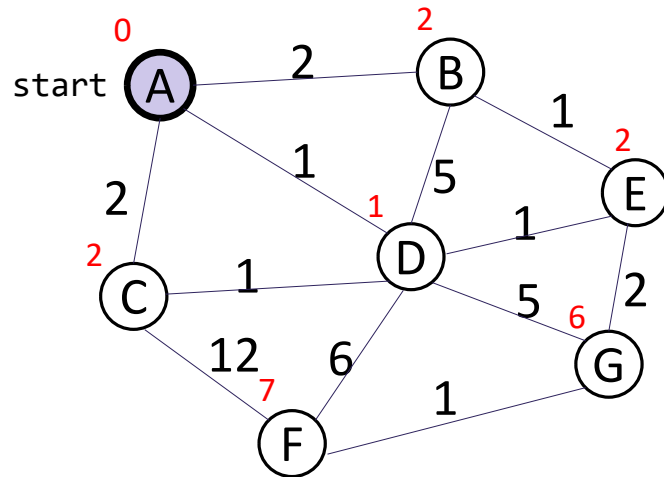
Dijkstra's Algorithm: Example #3 (undirected)



Order Added to
Known Set:
A, D,

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B		≤ 2	A
C		≤ 2	A
D	Y	≤ 1	A
E		≤ 2	D
F		≤ 7	D
G		≤ 6	D

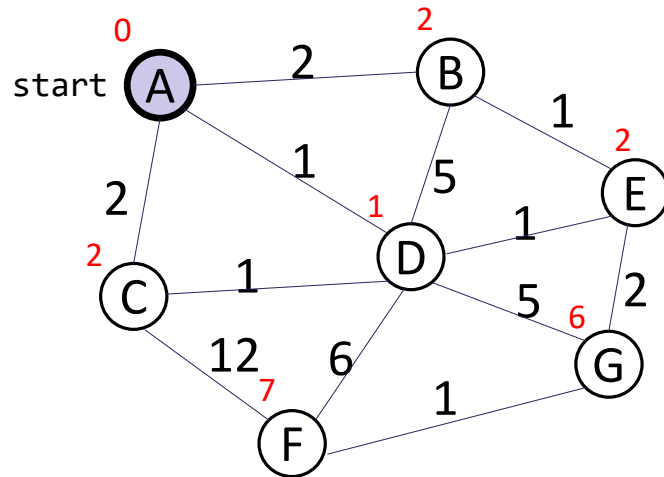
Dijkstra's Algorithm: Example #3 (undirected)



Order Added to
Known Set:
A, D, B,

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	≤ 2	A
C		≤ 2	A
D	Y	≤ 1	A
E		≤ 2	D
F		≤ 7	D
G		≤ 6	D

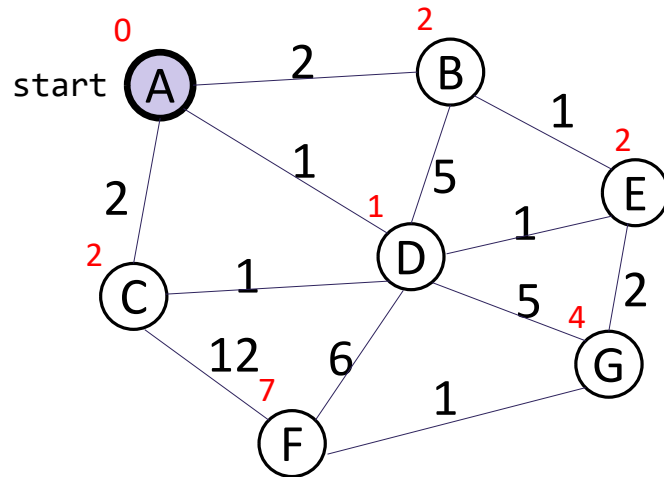
Dijkstra's Algorithm: Example #3 (undirected)



Order Added to
Known Set:
A, D, B, C,

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	≤ 2	A
C	Y	≤ 2	A
D	Y	≤ 1	A
E		≤ 2	D
F		≤ 7	C
G		≤ 6	D

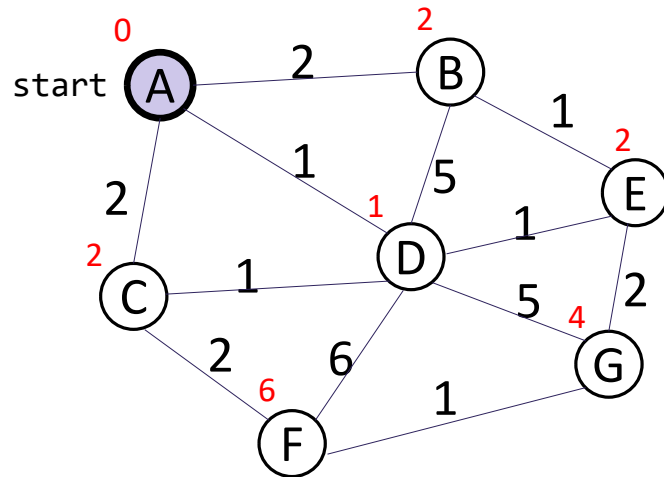
Dijkstra's Algorithm: Example #3 (undirected)



Order Added to
Known Set:
A, D, B, C, E

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	≤ 2	A
C	Y	≤ 2	A
D	Y	≤ 1	A
E	Y	≤ 2	D
F		≤ 7	C
G		≤ 4	E

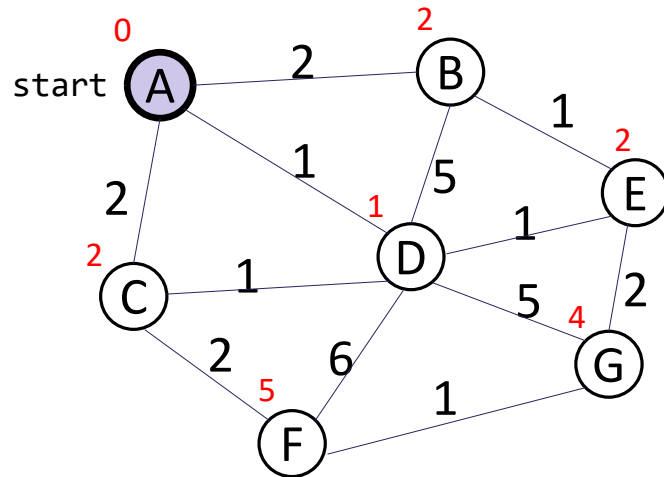
Dijkstra's Algorithm: Example #3 (undirected)



Order Added to
Known Set:
A, D, B, C, E, G

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	≤ 2	A
C	Y	≤ 2	A
D	Y	≤ 1	A
E	Y	≤ 2	D
F		≤ 6	G
G	Y	≤ 4	E

Dijkstra's Algorithm: Example #3 (undirected)

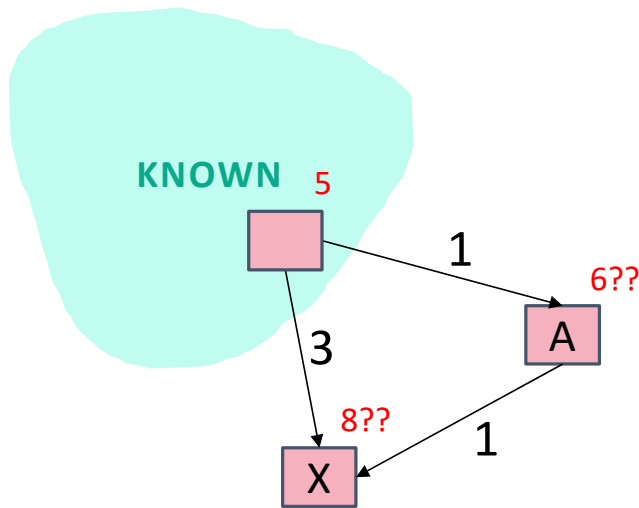


Order Added to
Known Set:
A, D, B, C, E, G, F

Vertex	Known?	distTo	edgeTo
A	Y	0	/
B	Y	≤ 2	A
C	Y	≤ 2	A
D	Y	≤ 1	A
E	Y	≤ 2	D
F	Y	≤ 5	G
G	Y	≤ 4	E

Path $\rightarrow$ A, D, E, G, F

Why Does Dijkstra's Work?



Example:

- We're about to add X to the known set
- But how can we be sure we won't later find a path through some node A that is shorter to X?

INVARIANT

Dijkstra's Algorithm Invariant

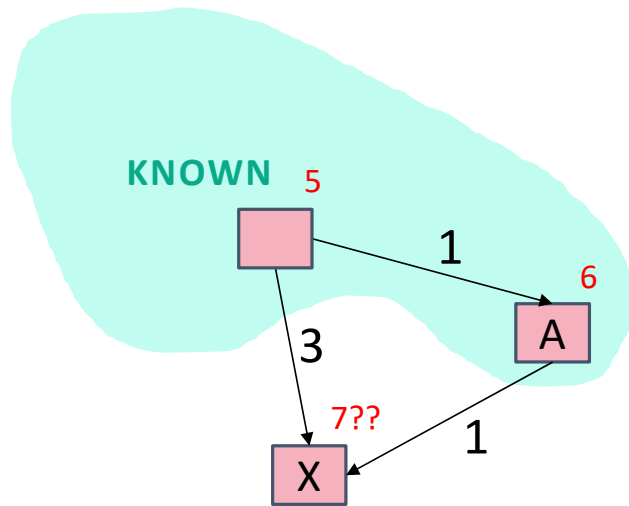
All vertices in the "known" set have the correct shortest path

- Similar "First Try Phenomenon" to BFS



- How can we be sure we won't find a shorter path to X later?

Why Does Dijkstra's Work?



Example:

- We're about to add X to the known set
- But how can we be sure we won't later find a path through some node A that is shorter to X?
- Because *if we could, Dijkstra's would explore A first*

INVARIANT

Dijkstra's Algorithm Invariant

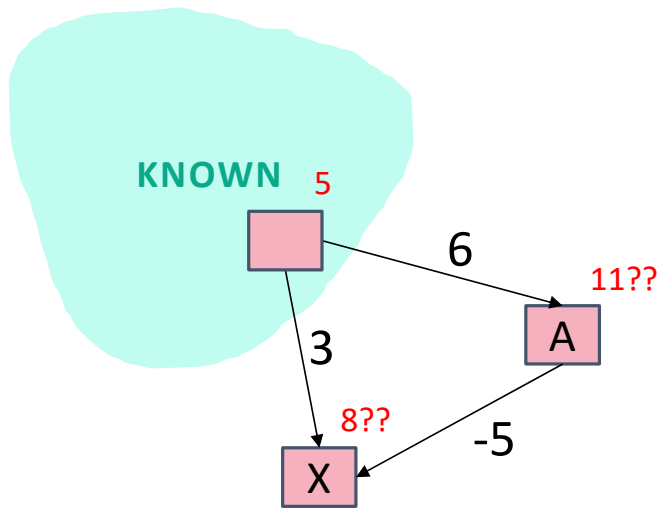
All vertices in the "known" set have the correct shortest path

- Similar "First Try Phenomenon" to BFS



- How can we be sure we won't find a shorter path to X later?
 - **Key Intuition:** Dijkstra's works because:
 - IF we always add the closest vertices to "known" first,
 - THEN by the time a vertex is added, any possible relaxing has happened and the path we know is *always the shortest!*

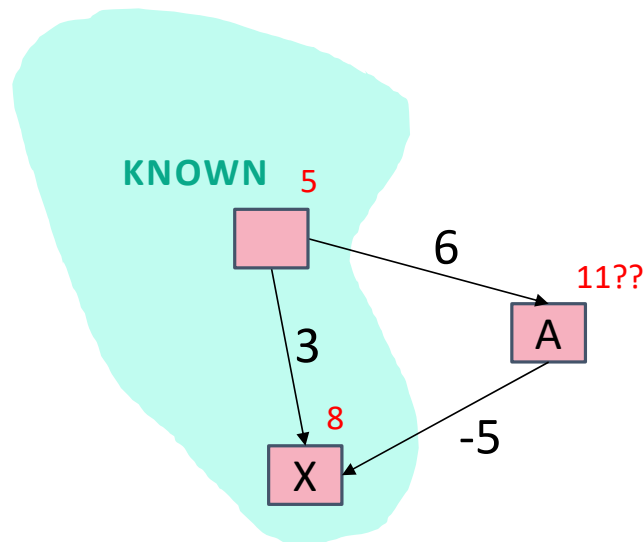
When *Doesn't* Dijkstra's Work?



Example:

- Which vertex do we add first?
 - X, using edge 3, because $8 < 11$

When *Doesn't* Dijkstra's Work?



Example:

- Which vertex do we add first?
 - X, using edge 3, because $8 < 11$
- Is 8 the correct shortest path length to X?
 - No! Going through A, we could have gotten a path of length 6

INVARIANT

Dijkstra's Algorithm Invariant

All vertices in the known set have the correct shortest path



- Dijkstra's Algorithm is not guaranteed to work on graphs with **negative edge weights**
 - It *can* work, but is fooled when a negative edge "hides" behind a large edge weight
 - Will still run, but give wrong answer

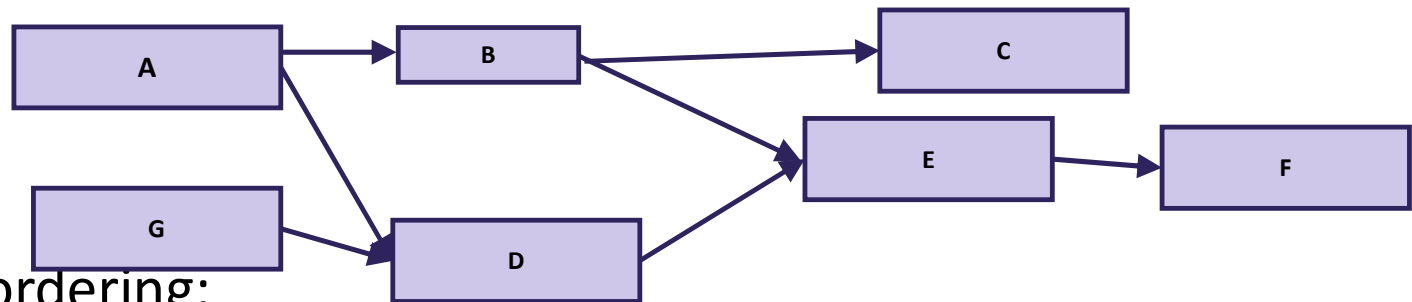
Lecture Outline

- *Review* DFS, BFS, Unweighted Shortest Paths
- Weighted Graphs
- Minimum Spanning Tree (MST)
 - Prim's Algorithm
 - Kruskal's Algorithm
- Weighted Shortest Path Problem
- Dijkstra's Algorithm
 - Definition & Examples
 - Implementing Dijkstra's
- Topological Sorting 

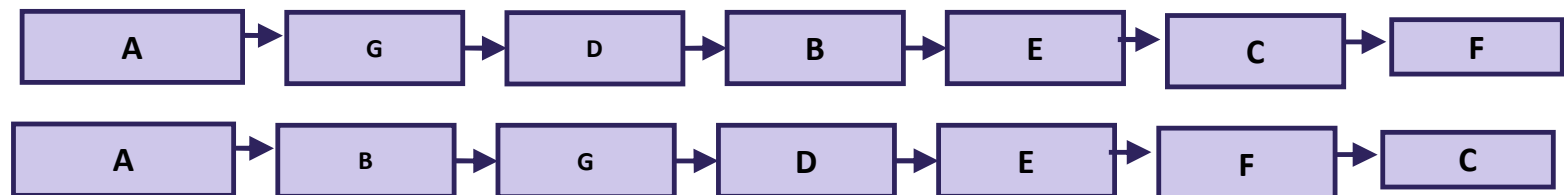
Topological Sort

Sorting Dependencies

- Given a set of courses and their prerequisites, find an order to take the courses in (assuming you can only take one course per quarter)



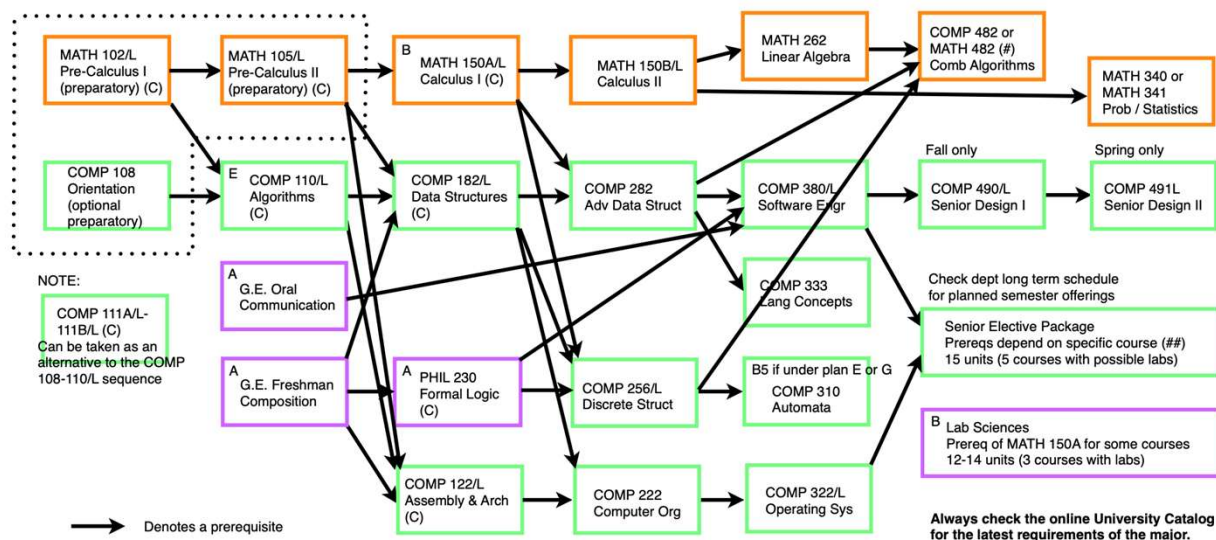
- Possible ordering:



CSU Northridge, Computer Science Department

COMPUTER SCIENCE MAJOR

2021-23 Catalog Requirements



Must receive "C-" or better in all courses attempted at CSUN that are required by major unless otherwise stated; courses marked with (C) require "C" or better when taken at CSUN. All transfer courses must be "C" or better. The "C" or better requirement in MATH 102/L, 105/L and 150A/L is to satisfy COMP 110/L, 122/L, 182/L and 282 prerequisites.

A B E

Denotes major requirement that also satisfies G.E. Section A, B or E.

.....

Placement in courses leading to major requirements is dependent on high school preparation, MPT scores and current computer programming skills. Consult with an academic advisor for proper placement.

(#) MATH 482 has as one of its prerequisites MATH 262; COMP 482 does not have MATH 262 as one of its prerequisites.

(##) Check senior electives for their prerequisites, many require just COMP 380/L, some require just COMP 322/L, some require both COMP 322/L and COMP 380/L, and a few require COMP 310.

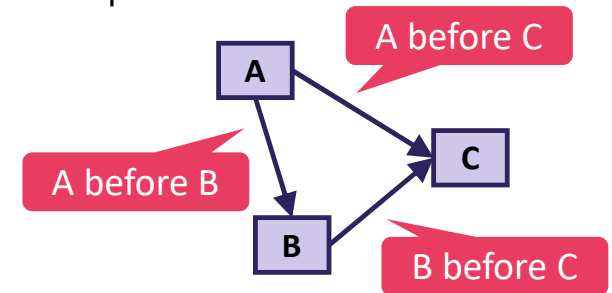
A Graduation Evaluation is required 2 to 3 semesters prior to planned date of graduation.

Always check the online University Catalog for the latest requirements of the major.

Topological Sort

- A **topological sort** of a directed graph G without cycles is an ordering of the nodes, where for every edge in the graph, the origin appears before the destination in the ordering
- Intuition: a “dependency graph”
 - An edge (u, v) means u must happen before v
 - A topological sort of a dependency graph gives an ordering that **respects dependencies**
- Applications:
 - Graduating
 - Compiling multiple Java files
 - Multi-job Workflows

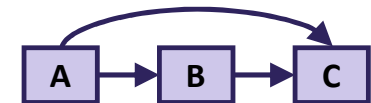
Input:



Topological Sort:

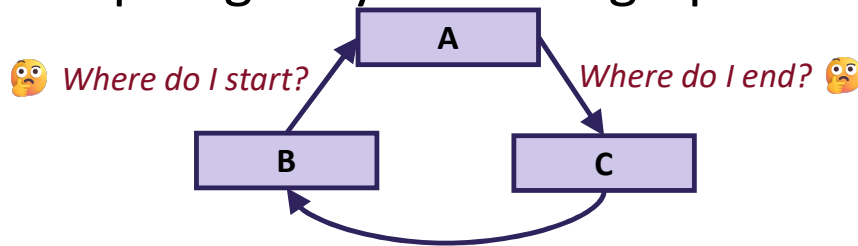


With original edges for reference:



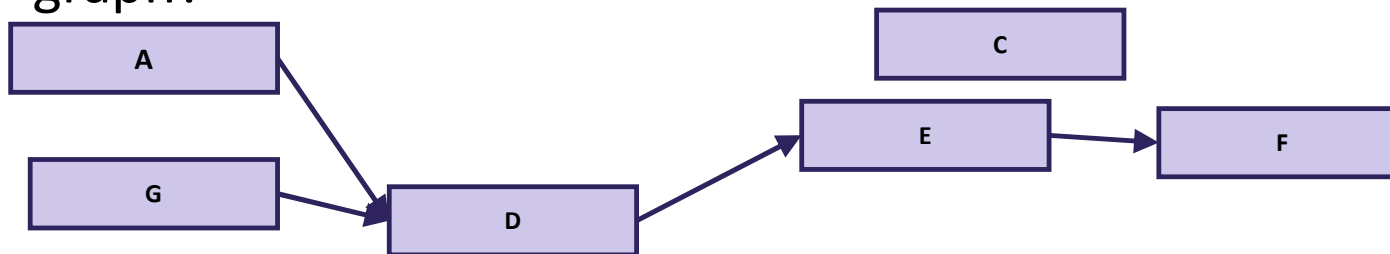
Can We Always Topo Sort a Graph?

- Can you topologically sort this graph?



No 🤔

- What's the difference between this graph and our first graph?



- A graph has a topological ordering iff it is a DAG
 - But a DAG can have multiple orderings

DIRECTED ACYCLIC GRAPH

- A **directed graph** without any **cycles**
- Edges may or may not be weighted

How To Perform Topo Sort?

- Topo sort is an ordering problem. Could we use... BFS?

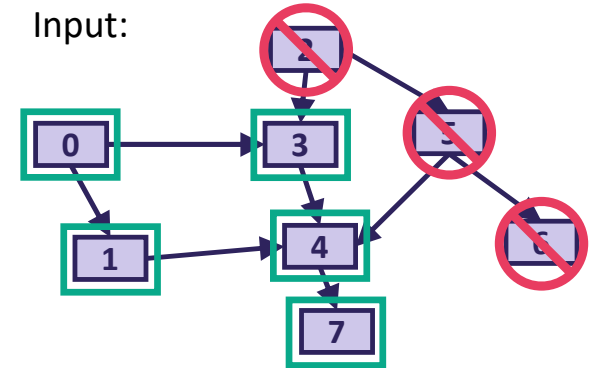
IDEA 1

Performing Topo Sort

Use BFS, starting from a vertex with no incoming edges

Doesn't reach all vertices ☹️

Input:



BFS starting at 0:



How To Perform Topo Sort?

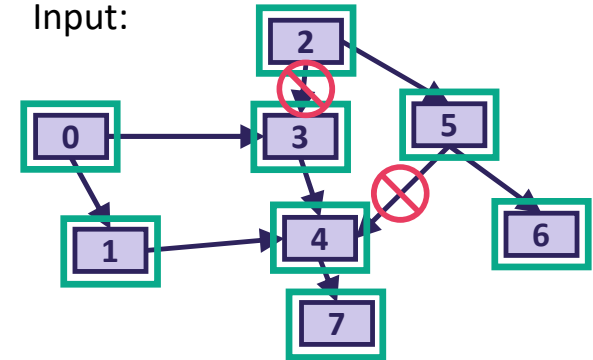
- Okay, there may be multiple “roots”. What if we use BFS multiple times?

IDEA 2

Performing Topo Sort

Use BFS, starting from ALL vertices with no incoming edges

Input:



BFS starting at 0:

+ BFS starting at 2:



How To Perform Topo Sort?

- Okay, there may be multiple “roots”. What if we use BFS multiple times?

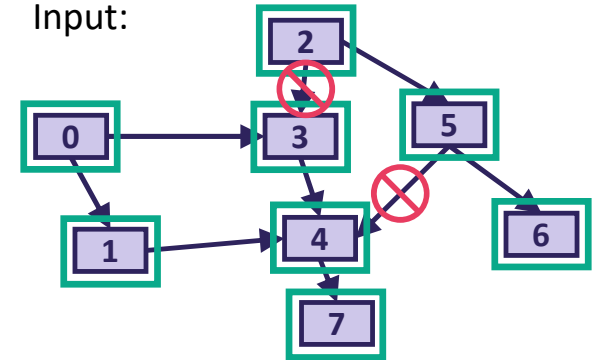
IDEA 2

Performing Topo Sort

Use BFS, starting from ALL vertices with no incoming edges

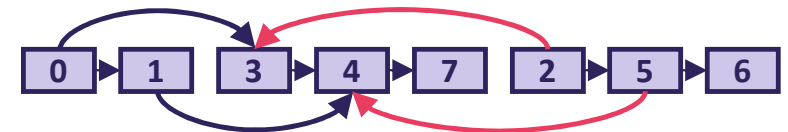
Doesn't respect all edges ☹️

Input:



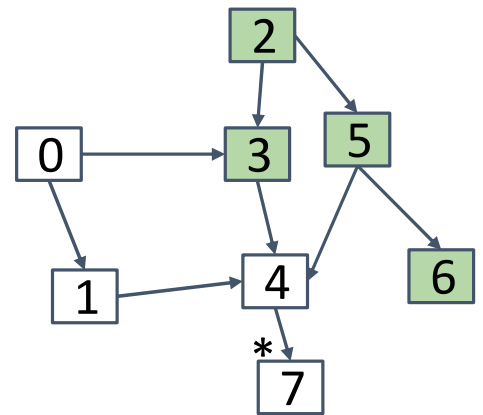
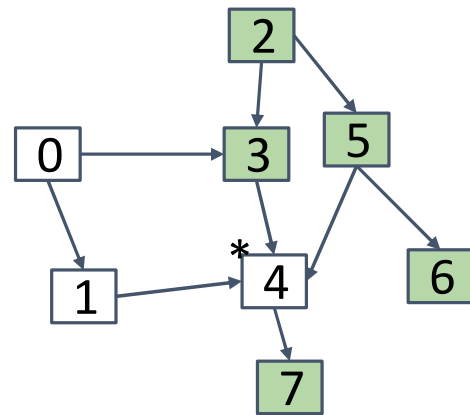
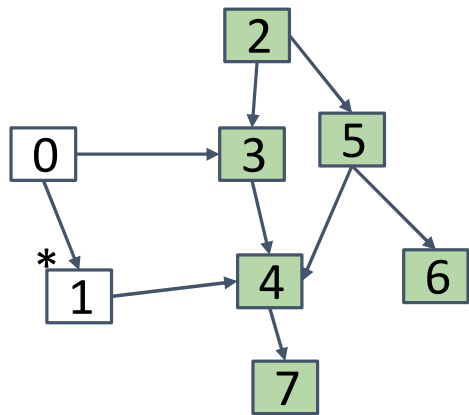
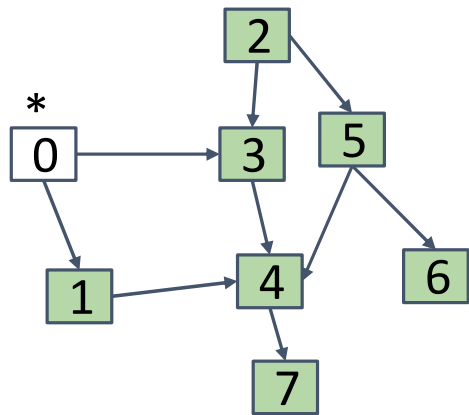
BFS starting at 0:

+ BFS starting at 2:



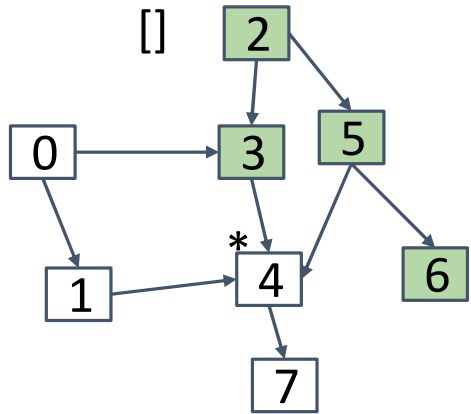
Idea: DFS:

- In DFS, we print a vertex and then recursively call DFS for its adjacent vertices. In topological sorting, we need to print a vertex before its adjacent vertices.
- We can modify DFS to find the Topological Sorting of a graph. In DFS,
 - We start from a vertex, we first print it, and then
 - Recursively call DFS for its adjacent vertices.
- In topological sorting,
 - We use a temporary stack.
 - We don't print the vertex immediately,
 - We first recursively call topological sorting for all its adjacent vertices, then push it to a stack.
 - Finally, print the contents of the stack.
- [Demo for topSort](#)



Postorder:

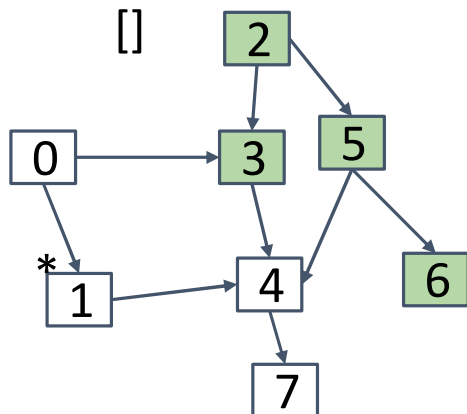
[]



Postorder: [7, 4]

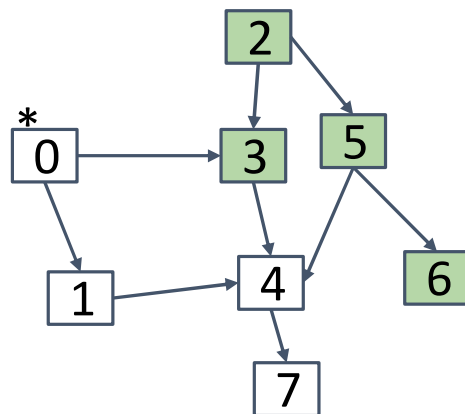
Postorder:

[]



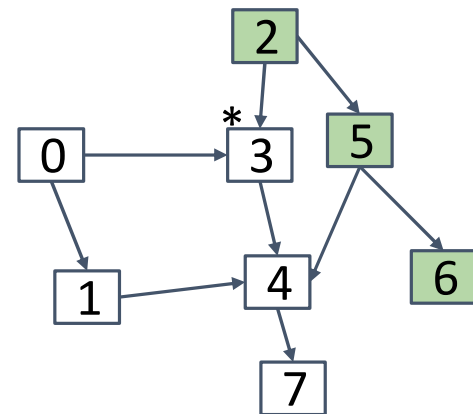
Postorder: [7, 4, 1]

Postorder: []



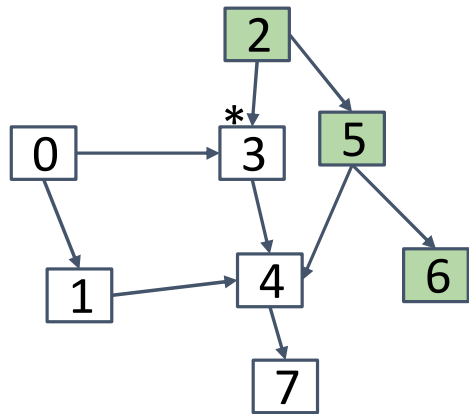
Postorder: [7, 4, 1]

Postorder: [7]

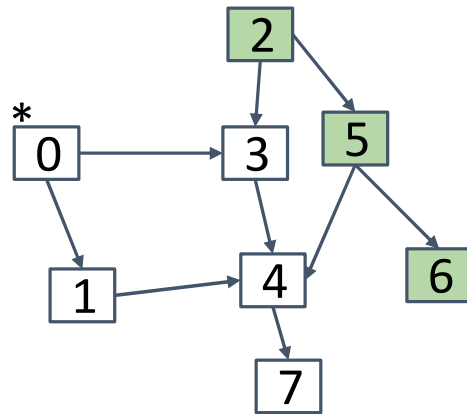


Postorder: [7, 4, 1, 3]

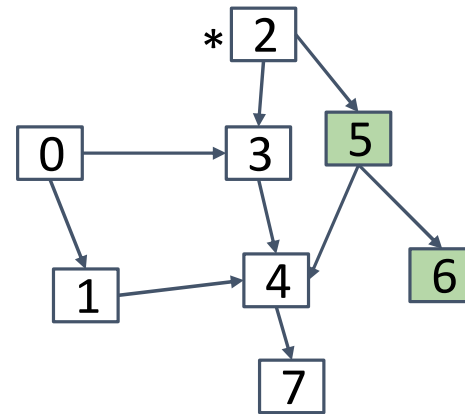
Topological Sort (Demo 2/2)



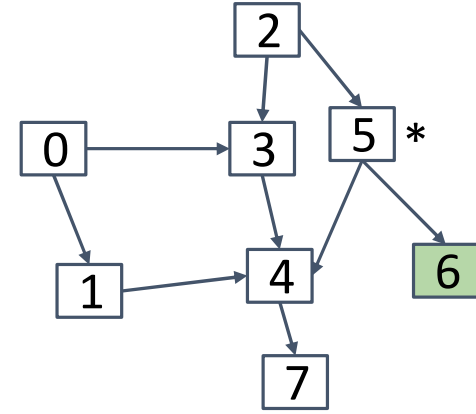
Postorder: [7, 4, 1, 3]



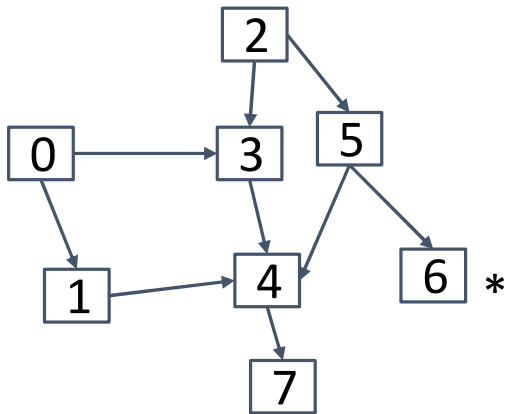
Postorder: [7, 4, 1, 3, 0]



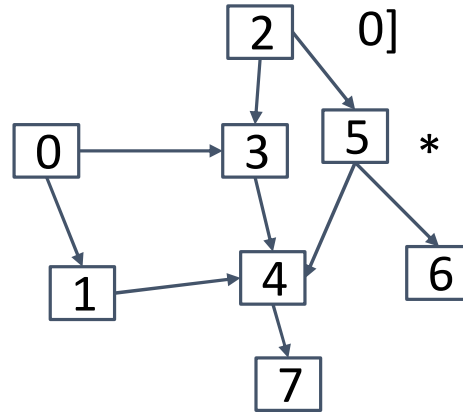
Postorder: [7, 4, 1, 3, 0]



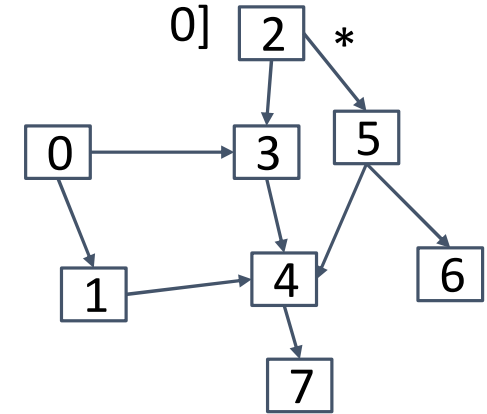
Postorder: [7, 4, 1, 3, 0]



Postorder: [7, 4, 1, 3, 0, 6]

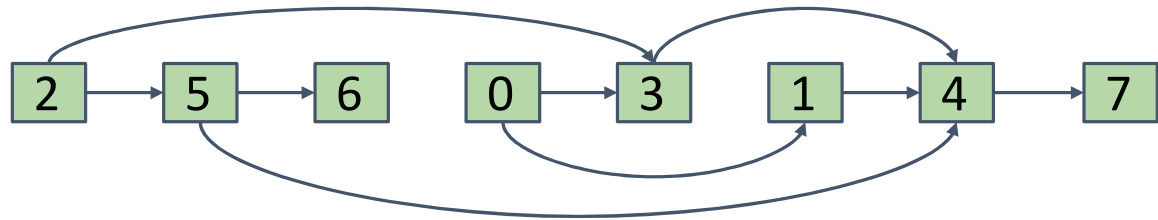
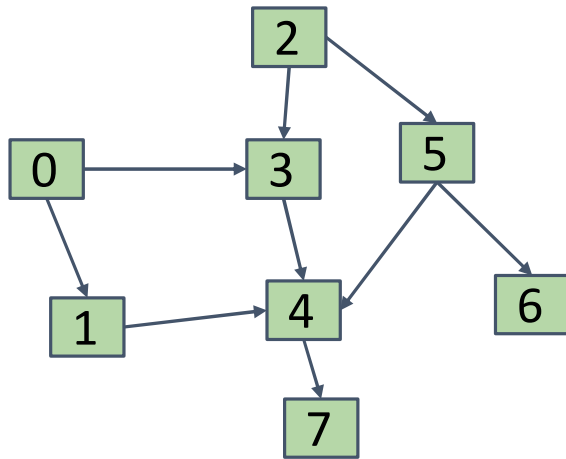


Postorder: [7, 4, 1, 3, 0, 6, 5]



Postorder: [7, 4, 1, 3, 0, 6, 5, 2]

Topological Sort

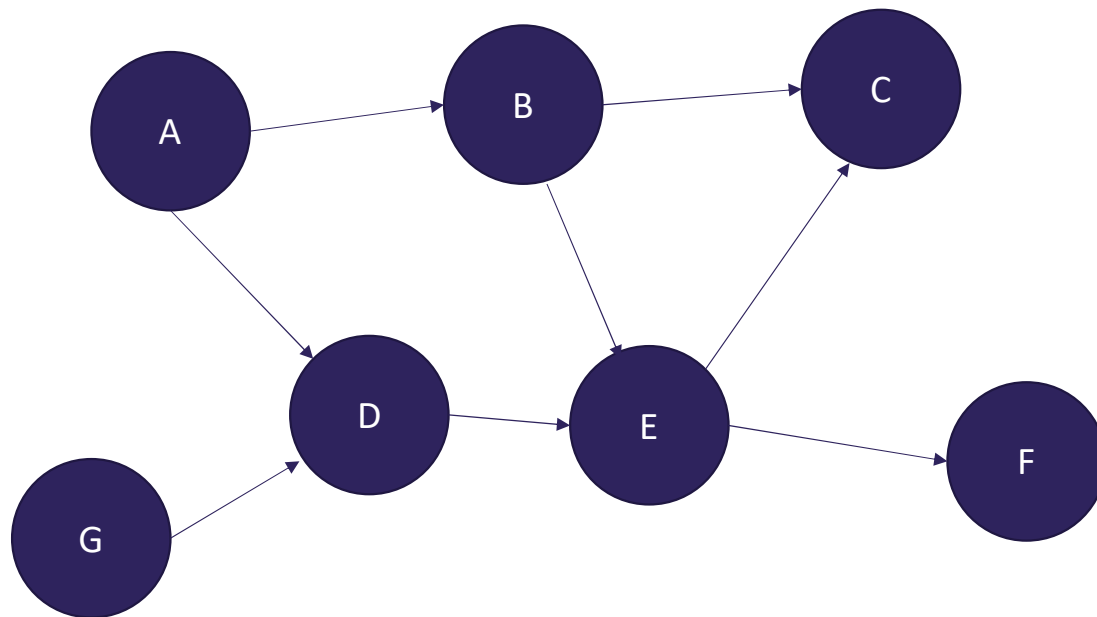


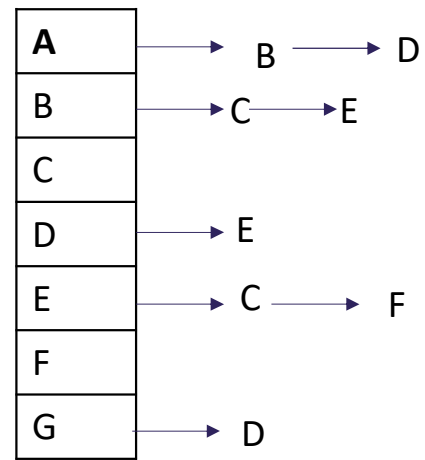
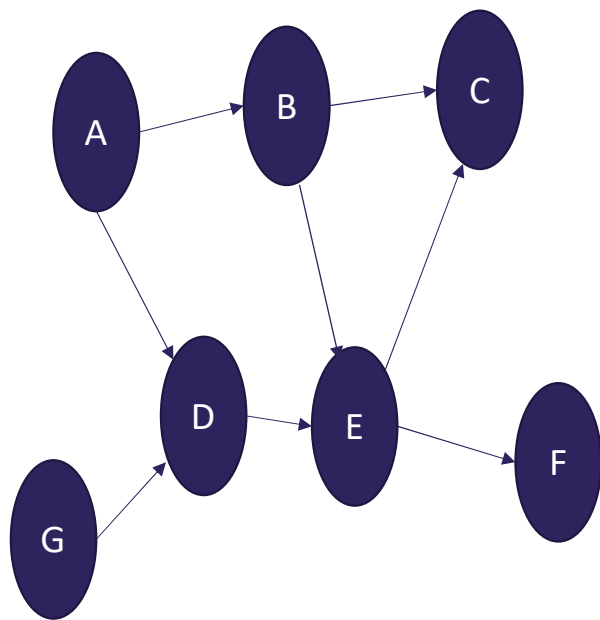
The reason it's called topological sort: Can think of this process as sorting our nodes so they appear in an order consistent with edges, e.g. [2, 5, 6, 0, 3, 1, 4, 7]

- When nodes are sorted in diagram, arrows all point rightwards.

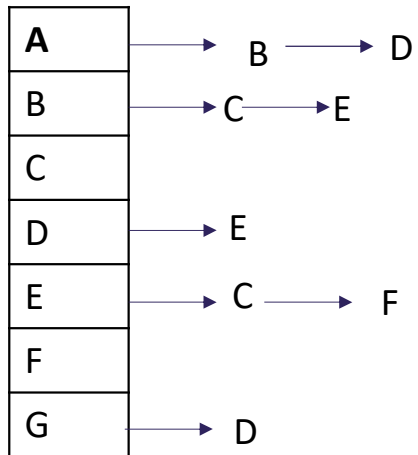
None Recursive Algorithm

1. Created visited list
2. While the visited list does not contain all the vertices
 - Find a vertex that has no successor
 - Remove all the edges that lead to it from edgeList
 - Add it to the visited list
 - Add it to the topological list

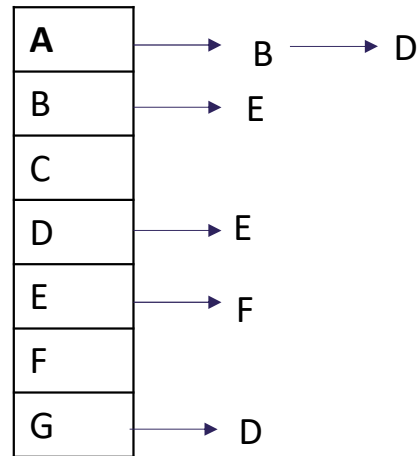




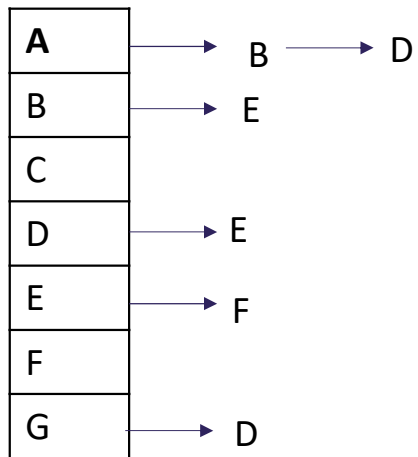
Loop 1: Vertex C is selected



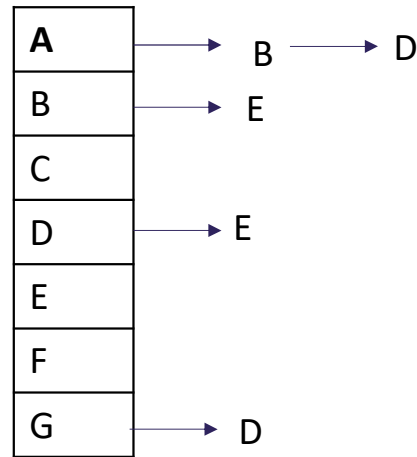
visitedList {C}
List {C}



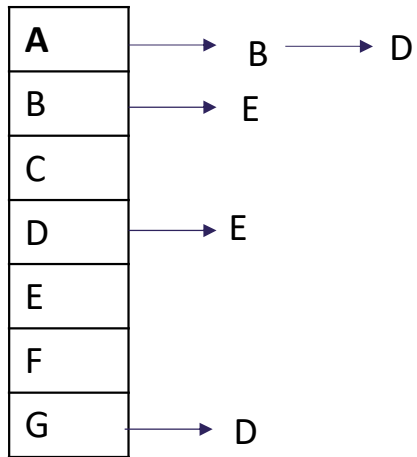
Loop 2: Vertex F is selected



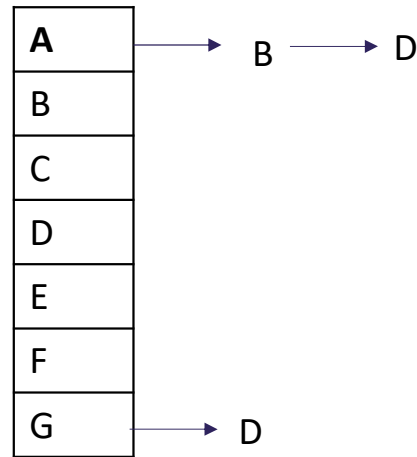
visitedList {C,F}
List {F,C}



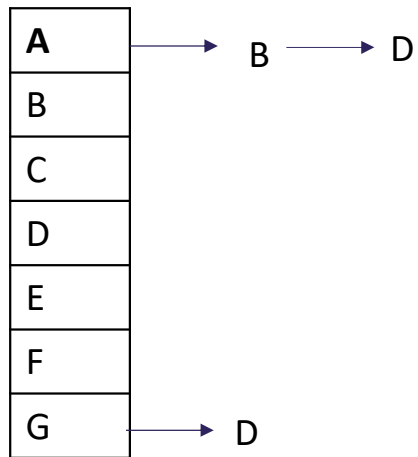
Loop 3: Vertex E is selected



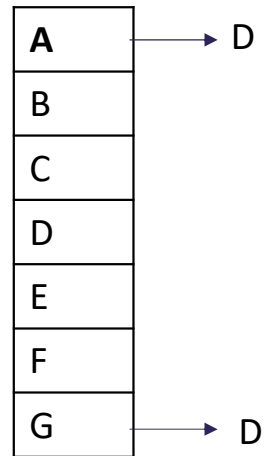
visitedList {C,F,E}
List {E,F,C}



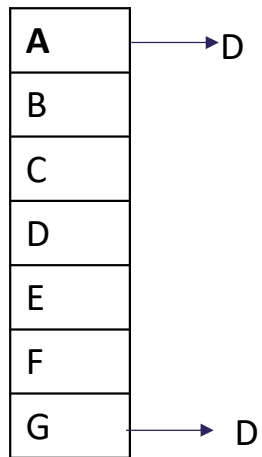
Loop 4: Vertex B is selected



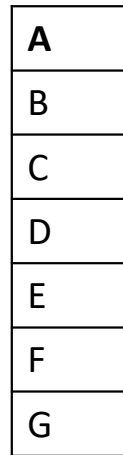
visitedList {C,F,E,B}
List {B,E,F,C}



Loop 5: Vertex D is selected



visitedList {C,F,E,B,D}
List {D,B,E,F,C}



Loop 6: Vertex A is selected

A
B
C
D
E
F
G

visitedList {C,F,E,B,D,A}
List {A,D,B,E,F,C}

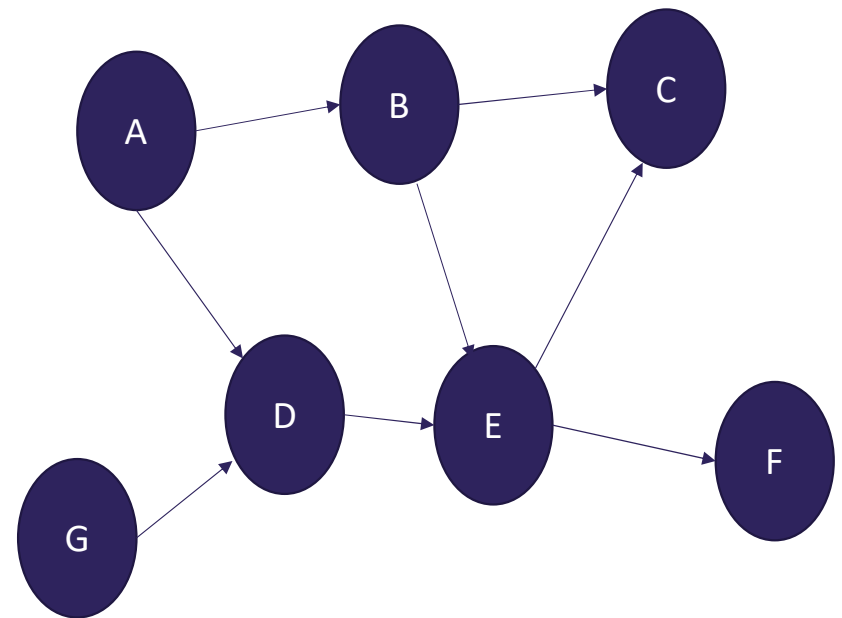
A
B
C
D
E
F
G

Loop 7: Vertex G is selected

A
B
C
D
E
F
G

visitedList {C,F,E,B,D,A,G}
List {G,A,D,B,E,F,C}

A
B
C
D
E
F
G



Topological Sorting: $G \rightarrow A \rightarrow D \rightarrow B \rightarrow E \rightarrow F \rightarrow C$

Opening Problem

- The Seven Bridges of Königsberg problem.
 - The Seven Bridges of Königsberg is a famous mathematical problem that involves finding a route across each of seven bridges in the city of Königsberg (now Kaliningrad) exactly once.
 - [Leonhard Euler](#) proved that such a route was impossible, laying the foundation for graph theory.
 - [Eulerian Walk](#)